

Iksung.Reader SDK — 통합 사용 설명서

버전: SDK V1.0 / .NET 8 + .NET Framework 4.7.2 지원 **문서 빌드일:** 2026-05-13 **대상:** 익성전자 NFC/RFID 리더기 응용 프로그램 개발자 **채널 지원:** Serial (UART 115200 bps) · USB CCID (PC/SC) · TCP/IP Socket

머리말

본 문서는 **Iksung.Reader .NET SDK**의 통합 사용 설명서입니다. 모든 카드 표준 / 시스템 기능 / 예외 처리 / API 레퍼런스를 단일 문서로 정리했으며, 이전 [IS_3400_V3.0_Reader-C#_Library_V3.2.pdf](#) (D2xx 기반 단종 라이브러리)의 후속 문서입니다.

항목	이전 (V3.2 / D2xx)	신규 (Iksung.Reader SDK)
인터페이스	FTDI D2xx 만 지원	Serial / TCP Socket / PC/SC 3개 채널 추상화
비동기	동기 호출 + 콜백	async/await 전체 지원
패키지	직접 DLL 참조	NuGet Iksung.Reader
플랫폼	Windows .NET Framework 만	.NET 8 (크로스 플랫폼) + .NET 4.7.2
예외 처리	반환값 검사	IksungProtocolException / IksungTimeoutException / ChannelDisconnectedException
자동 감지	폴링 루프 직접 작성	TagDetected 이벤트 + AutoReconnect
시스템 진단	별도 도구 필요	PingAsync / GetReaderInfoAsync / Raw 패킷 로그

예제 프로젝트 생성 및 실행 가이드

이 문서는 **Iksung.Reader SDK** 예제를 처음 접하는 개발자가
직접 프로젝트를 만들거나 제공된 예제를 실행할 수 있도록 단계별로 안내합니다.

목차

1. 사전 준비
2. 예제 소스코드 다운로드
3. 제공된 예제 열고 실행하기
4. 신규 NET 8 프로젝트 만들기 — Visual Studio
5. 신규 NET 8 프로젝트 만들기 — CLI
6. 신규 NET Framework 4.7.2 프로젝트 만들기
7. SDK 패키지 추가하기
8. 리더기 포트 확인 방법
9. 첫 번째 코드 작성 및 실행
10. 자주 묻는 질문 및 오류 해결

1. 사전 준비

1-1. 필수 소프트웨어

아래 소프트웨어가 PC에 설치되어 있어야 합니다.

소프트웨어	버전	다운로드
Visual Studio 2022	17.x 이상	https://visualstudio.microsoft.com
.NET SDK	8.0 이상	https://dotnet.microsoft.com/download
Git	최신	https://git-scm.com

Visual Studio 설치 시 워크로드 선택:

.NET 데스크톱 개발 워크로드를 반드시 체크합니다.

(NET Framework 4.7.2 예제 빌드에 필요)

[이미지 자리]

Visual Studio Installer 화면 — ".NET 데스크톱 개발" 워크로드가 체크된 상태

1-2. .NET SDK 설치 확인

터미널(명령 프롬프트 또는 PowerShell)을 열고 아래 명령을 실행합니다.

```
dotnet --version
```

출력 예시:

```
8.0.xxx
```

버전이 **8.0** 이상이면 준비 완료입니다.

1-3. 하드웨어 연결

리더기를 USB 케이블 또는 RS-232 케이블로 PC에 연결합니다.

[이미지 자리]

IS-NFC 리더기를 USB 케이블로 PC에 연결한 사진

연결 후 **Windows 장치 관리자**에서 포트 번호를 확인합니다 (8장 참조).

2. 예제 소스코드 다운로드

2-1. GitHub에서 클론

터미널을 열고 원하는 폴더로 이동한 뒤 아래 명령을 실행합니다.

```
git clone https://github.com/Michael202301/ISReaderPro.Sdk.git
```

클론이 완료되면 **ISReaderPro.Sdk** 폴더가 생성됩니다.

```
ISReaderPro.Sdk/  
├── NET8-Samples/           ← .NET 8 예제 (14개)  
│   ├── HelloWorld.Console/  
│   ├── 01-ReadAnyUid/  
│   ├── 02-Iso14443a/  
│   └── ...  
├── NET4x-Samples/         ← .NET Framework 4.7.2 예제 (14개)  
│   ├── HelloWorld.Console/  
│   ├── 01-ReadAnyUid/  
│   └── ...  
├── src/  
│   └── Iksung.Reader/      ← SDK 소스 (빌드 대상)  
└── docs/                  ← 이 문서 포함
```

2-2. ZIP으로 다운로드 (Git 미설치 시)

GitHub 저장소 페이지에서 **Code → Download ZIP** 버튼을 클릭하여 ZIP 파일을 받은 뒤 원하는 폴더에 압축을 해제합니다.

[이미지 자리]

GitHub 저장소 페이지에서 "Code" 초록 버튼 클릭 → "Download ZIP" 메뉴가 보이는 화면

3. 제공된 예제 열고 실행하기

3-1. Visual Studio 2022로 솔루션 열기

1. Visual Studio 2022를 실행합니다.
2. 시작 화면에서 **"프로젝트 또는 솔루션 열기"** 를 클릭합니다.

 [이미지 자리]

Visual Studio 2022 시작 화면 — "프로젝트 또는 솔루션 열기" 버튼이 강조된 모습

3. 파일 탐색기에서 클론한 폴더로 이동하여

`ISReaderPro.Sdk.slnx` 파일을 선택하고 **열기**를 클릭합니다.

 [이미지 자리]

파일 탐색기에서 `ISReaderPro.Sdk.slnx` 파일을 선택한 화면

4. 솔루션이 열리면 **솔루션 탐색기**에 두 폴더가 보입니다.

 [이미지 자리]

Visual Studio 솔루션 탐색기 — `NET8-Samples` 와 `NET4x-Samples` 폴더 트리가 펼쳐진 모습

3-2. 실행할 예제 프로젝트 선택

1. 솔루션 탐색기에서 실행하려는 예제 프로젝트를 **우클릭**합니다.

(예: `NET8-Samples` → `01-ReadAnyUid`)

2. "**시작 프로젝트로 설정**" 을 클릭합니다.

 [이미지 자리]

솔루션 탐색기에서 `ReadAnyUid` 프로젝트를 우클릭 → "시작 프로젝트로 설정" 메뉴가 보이는 화면

3-3. 포트 인수 설정 후 실행

예제는 실행 시 COM 포트를 인수로 받습니다.

Visual Studio에서 인수를 설정하려면:

1. 프로젝트를 우클릭 → "속성" 클릭
2. "디버그" 탭 → "일반" → "명령줄 인수" 에 포트 번호 입력

예: COM3

[이미지 자리]

프로젝트 속성 창 — 디버그 탭의 "명령줄 인수" 입력란에 COM3 이 입력된 화면

3. F5 키 또는 상단 ▶ 실행 버튼을 눌러 실행합니다.

[이미지 자리]

콘솔 창에 [IKSUNG] Firmware: V1.x.x 와 UID 출력이 보이는 실행 결과 화면

3-4. dotnet CLI로 바로 실행 (Visual Studio 없이)

터미널에서 클론한 폴더로 이동한 뒤 아래 명령을 실행합니다.

```
# .NET 8 예제 실행 (Windows COM3 기준)
dotnet run --project NET8-Samples/01-ReadAnyUid -- COM3

# .NET 8 예제 실행 (Linux)
dotnet run --project NET8-Samples/01-ReadAnyUid -- /dev/ttyUSB0

# .NET Framework 4.7.2 예제 실행
dotnet run --project NET4x-Samples/01-ReadAnyUid -- COM3
```

-- 뒤의 값이 프로그램에 전달되는 인수입니다 (포트 이름).

4. 신규 NET 8 프로젝트 만들기 — Visual Studio 2022

SDK를 이용해 처음부터 직접 프로젝트를 만드는 방법입니다.

4-1. 새 프로젝트 만들기

1. Visual Studio 2022를 실행합니다.
2. "새 프로젝트 만들기" 를 클릭합니다.

[이미지 자리]

Visual Studio 2022 시작 화면 — "새 프로젝트 만들기" 버튼

3. 검색창에 **콘솔** 을 입력하고
"콘솔 앱" (C#, .NET / .NET Core용) 을 선택합니다.
 **"콘솔 앱 (.NET Framework)"** 와 다릅니다. 올바른 항목을 선택하세요.

[이미지 자리]

새 프로젝트 대화 상자 — 검색창에 "콘솔" 입력 후 C# "콘솔 앱"(비 .NET Framework)이 선택된 화면

4. **"다음"** 클릭 후 프로젝트 이름과 저장 위치를 설정합니다.

항목	예시 값
프로젝트 이름	MyNfcApp
위치	C:\Projects\
솔루션 이름	MyNfcApp

5. **"다음"** 클릭 후 프레임워크를 선택합니다.

[이미지 자리]

프레임워크 선택 화면 — **.NET 8.0 (장기 지원)** 이 선택된 드롭다운

6. "만들기" 를 클릭합니다.

4-2. SDK 패키지 추가

7장 SDK 패키지 추가하기를 참조하세요.

4-3. Program.cs 코드 작성

생성된 `Program.cs` 파일을 열고 내용을 아래와 같이 작성합니다.

```
using Iksung.Reader;
using Iksung.Reader.Exceptions;

string portName = args.Length > 0 ? args[0] : "COM3";

Console.WriteLine($"[IKSUNG] Connecting to {portName}...");

await using var reader = await IksungReader.ConnectSerialAsync(portName);
Console.WriteLine($"[IKSUNG] Firmware: {await reader.ReadVersionAsync()}");
Console.WriteLine("[IKSUNG] Place a card. Press Ctrl+C to exit.\n");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] uid = await reader.ReadIso14443aUidAsync(500, cts.Token);
        Console.WriteLine($"{DateTime.Now:HH:mm:ss}   UID: {BitConverter.ToString(uid).Replace("-", "")}");
    }
    catch (IksungTimeoutException) { /* 카드 없음 - 대기 */ }
    catch (IksungProtocolException) { /* 인식 실패 - 대기 */ }
    catch (OperationCanceledException){ break; }

    try { await Task.Delay(300, cts.Token); }
    catch (OperationCanceledException){ break; }
}

Console.WriteLine("\n[IKSUNG] Done.");
```

5. 신규 NET 8 프로젝트 만들기 — .NET CLI

Visual Studio 없이 터미널만으로 프로젝트를 생성하는 방법입니다.

5-1. 프로젝트 생성

```
# 프로젝트 폴더 생성 및 이동
mkdir MyNfcApp
cd MyNfcApp

# 콘솔 앱 생성 (.NET 8)
dotnet new console -f net8.0
```

생성 결과:

```
MyNfcApp/
├─ MyNfcApp.csproj
└─ Program.cs
```

5-2. SDK 패키지 추가

```
dotnet add package Iksung.Reader
```

패키지가 NuGet에 아직 게시되지 않은 경우 [7-2장 로컬 참조](#) 방법을 사용합니다.

5-3. 코드 작성 및 실행

```
# Program.cs 편집 (원하는 에디터 사용)
code Program.cs      # VS Code
notepad Program.cs   # 메모장

# 실행
dotnet run -- COM3
```

6. 신규 NET Framework 4.7.2 프로젝트 만들기

레거시 WinForms / WPF 앱에 SDK를 통합하거나
.NET Framework 환경에서 예제를 실행하려는 경우 이 방법을 사용합니다.

6-1. Visual Studio에서 신규 생성

1. "새 프로젝트 만들기" 클릭
2. 검색창에 **콘솔** 입력 →
"콘솔 앱 (.NET Framework)" (C#) 선택

 [이미지 자리]

새 프로젝트 대화 상자 — "콘솔 앱 (.NET Framework)" 항목이 선택된 화면

3. "다음" 클릭 후 이름·위치 설정

4. 프레임워크 드롭다운에서 **.NET Framework 4.7.2** 선택

 [이미지 자리]

프레임워크 드롭다운 — ".NET Framework 4.7.2" 가 선택된 화면

5. "만들기" 클릭

6-2. .csproj 파일 설정 확인

생성된 **.csproj** 파일을 열어 아래 항목을 확인·추가합니다.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net472</TargetFramework>
    <LangVersion>latest</LangVersion>      <!-- C# 최신 문법 활성화 -->
    <Nullable>enable</Nullable>
    <ImplicitUsings>disable</ImplicitUsings> <!-- using 명시 필요 -->
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Iksung.Reader" Version="0.*" />
  </ItemGroup>
</Project>
```

LangVersion=latest 를 설정하면 .NET Framework 4.7.2에서도 C# 12 문법(컬렉션 표현식, switch 식 등)을 사용할 수 있습니다.

6-3. Program.cs 코드 작성 (.NET 4.x 스타일)

.NET Framework 4.x에서는 **Top-level statements**를 사용할 수 없습니다.
반드시 클래스와 `Main` 메서드를 명시해야 합니다.

```
using System;
using System.Threading;
using System.Threading.Tasks;
using Iksung.Reader;
using Iksung.Reader.Exceptions;

namespace MyNfcApp
{
    internal class Program
    {
        static async Task Main(string[] args)
        {
            string portName = args.Length > 0 ? args[0] : "COM3";

            Console.WriteLine($"[IKSUNG] Connecting to {portName}...");

            var reader = await IksungReader.ConnectSerialAsync(portName);
            try
            {
                Console.WriteLine($"[IKSUNG] Firmware: {await reader.ReadVersionAsync()}");
                Console.WriteLine("[IKSUNG] Place a card. Press Ctrl+C to exit.\n");

                using (var cts = new CancellationTokenSource())
                {
                    Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel();

                    while (!cts.Token.IsCancellationRequested)
                    {
                        try
                        {
                            byte[] uid = await reader.ReadIso14443aUidAsync(500, cts.Token);
                            Console.WriteLine(
                                $"{DateTime.Now:HH:mm:ss}   UID: " +
                                $"{BitConverter.ToString(uid).Replace("-", "")}");
                        }
                        catch (IksungTimeoutException) { }
                        catch (IksungProtocolException) { }
                        catch (OperationCanceledException){ break; }

                        try { await Task.Delay(300, cts.Token); }
                        catch (OperationCanceledException){ break; }
                    }
                }

                Console.WriteLine("\n[IKSUNG] Done.");
            }
            finally
            {
                await reader.DisposeAsync(); // await using 대신 명시적 해제
            }
        }
    }
}
```

```
}  
}
```

.NET 4.x와 .NET 8의 문법 차이 전체 목록은
.NET Framework 4.x 가이드를 참조하세요.

7. SDK 패키지 추가하기

7-1. NuGet 패키지로 추가 (게시 후 권장)

Visual Studio — NuGet 패키지 관리자 사용

1. 솔루션 탐색기에서 프로젝트를 **우클릭**
2. **"NuGet 패키지 관리"** 클릭

[이미지 자리]

솔루션 탐색기에서 프로젝트 우클릭 → "NuGet 패키지 관리" 메뉴가 표시된 화면

3. **"찾아보기"** 탭에서 **Iksung.Reader** 검색

[이미지 자리]

NuGet 패키지 관리자 — 검색창에 "Iksung.Reader" 입력 후 패키지 목록에 결과가 나온 화면

4. **Iksung.Reader** 선택 → 오른쪽 **"설치"** 버튼 클릭

[이미지 자리]

NuGet 패키지 관리자 — Iksung.Reader 패키지를 선택하고 "설치" 버튼이 보이는 화면

dotnet CLI 사용

```
dotnet add package Iksung.Reader
```

7-2. 로컬 프로젝트 참조 (현재 개발 중 권장)

NuGet에 아직 게시되지 않은 경우, SDK 소스를 직접 참조합니다.

Visual Studio — 프로젝트 참조 추가

1. 솔루션 탐색기에서 내 프로젝트 우클릭
2. "추가" → "프로젝트 참조" 클릭

[이미지 자리]

솔루션 탐색기 우클릭 메뉴 — "추가 → 프로젝트 참조" 항목이 보이는 화면

3. "찾아보기" 탭 → SDK 폴더로 이동 →

`src/Iksung.Reader/Iksung.Reader.csproj` 선택 → "확인"

[이미지 자리]

참조 관리자 대화 상자 — `Iksung.Reader.csproj` 파일이 선택된 상태

`.csproj` 파일 직접 편집

`.csproj` 파일을 텍스트 에디터로 열어 아래 내용을 추가합니다.

(`../..` 부분은 SDK 소스 폴더와의 **상대 경로**에 맞게 조정하세요.)

```
<ItemGroup>
  <!-- SDK 소스가 상위 폴더에 있는 경우 -->
  <ProjectReference Include="..\..\src\Iksung.Reader\Iksung.Reader.csproj" />
</ItemGroup>
```

dotnet CLI — 참조 추가

```
dotnet add reference ../../src/Iksung.Reader/Iksung.Reader.csproj
```

8. 리더기 포트 확인 방법

8-1. Windows — 장치 관리자

1. Win + X → "장치 관리자" 클릭
또는 검색창에 장치 관리자 입력 후 실행

[이미지 자리]

Windows 장치 관리자 창 — "포트(COM 및 LPT)" 섹션이 펼쳐져 있고,
USB Serial Port (COM3) 같은 항목이 보이는 화면

2. "포트 (COM 및 LPT)" 섹션을 펼칩니다.
3. 리더기가 연결된 항목에서 포트 번호를 확인합니다.
예: USB Serial Port (COM3) → 포트 이름: COM3

리더기를 뽑았다 꽂으면 목록이 사라졌다가 다시 나타나는 항목이 리더기입니다.

8-2. Linux

```
# 연결된 시리얼 장치 목록 확인
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null

# 또는 dmesg로 최근 연결 장치 확인
dmesg | grep tty | tail -5
```

출력 예시:

```
/dev/ttyUSB0
```

포트 접근 권한이 없으면 아래 명령으로 사용자를 dialout 그룹에 추가합니다.

```
sudo usermod -a -G dialout $USER
# 로그아웃 후 재로그인 필요
```

8-3. macOS

```
ls /dev/cu.usbserial-*
```

출력 예시:

```
/dev/cu.usbserial-1234
```

9. 첫 번째 코드 작성 및 실행

9-1. 빌드

Visual Studio

빌드 → **솔루션 빌드** (**Ctrl + Shift + B**)

[이미지 자리]

Visual Studio 출력 창 — "빌드: 1 성공, 0 실패, 0 경고" 메시지가 보이는 화면

dotnet CLI

```
dotnet build
```

성공 출력 예시:

```
빌드했습니다.
  경고 0개
  오류 0개
```


9-2. 실행

Visual Studio에서 실행

- **F5** : 디버그 모드로 실행 (중단점 사용 가능)
- **Ctrl + F5** : 디버그 없이 실행 (더 빠름)

dotnet CLI에서 실행

```
# Windows
dotnet run -- COM3

# Linux
dotnet run -- /dev/ttyUSB0

# macOS
dotnet run -- /dev/cu.usbserial-1234
```

9-3. 정상 실행 결과

리더기가 올바르게 연결된 경우 아래와 같이 출력됩니다.

```
[IKSUNG] Connecting to COM3...
[IKSUNG] Firmware: V1.2.3
[IKSUNG] Place a card. Press Ctrl+C to exit.
```

이후 카드를 리더기 위에 올려놓으면:

```
14:23:05  UID: 04A3F7C2
14:23:08  UID: 04A3F7C2
```

[이미지 자리]

콘솔 창에 펌웨어 버전과 UID가 출력된 실행 결과 화면

Ctrl + C 를 누르면 프로그램이 종료됩니다.

```
^C
[IKSUNG] Done.
```

10. 자주 묻는 질문 및 오류 해결

❌ `'IksungReader' does not exist in the current context`

원인: SDK 패키지가 설치되지 않았거나 `using Iksung.Reader;` 가 누락되었습니다.

해결: 1. NuGet 패키지 설치 확인 (7장) 2. 파일 상단에 `using Iksung.Reader;` 추가 3. `dotnet restore` 실행 후 재빌드

❌ `'await' requires that the method be marked async`

원인: .NET Framework 4.x에서 `Main` 메서드에 `async` 가 없습니다.

해결: `Main` 선언을 아래와 같이 수정합니다.

```
// ❌ 잘못된 예
static void Main(string[] args) { ... }

// ✅ 올바른 예
static async Task Main(string[] args) { ... }
```

❌ `System.IO.IOException: The port 'COM3' does not exist`

원인: 지정한 포트 이름이 잘못되었거나 리더기가 연결되어 있지 않습니다.

해결: 1. 리더기 USB 케이블이 완전히 꽂혀 있는지 확인 2. 8장을 참조해 정확한 포트 이름 재확인 3. 다른 프로그램이 해당 포트를 사용 중인지 확인 (다른 터미널·시리얼 모니터 종료)

❌ `Access to the port 'COM3' is denied`

원인: 다른 프로세스가 포트를 이미 점유 중이거나 권한 부족입니다.

해결: - 다른 시리얼 통신 프로그램(터미널, ISReaderPro 등)을 모두 닫은 뒤 재시도 - Linux의 경우 `diabloud` 그룹 추가 후 재로그인

✖ `IksungTimeoutException` 이 계속 발생하고 UID가 읽히지 않음

원인: 카드가 리더기 인식 범위 밖에 있거나 카드 타입이 맞지 않습니다.

해결: - 카드를 리더기 중앙에 완전히 밀착시켜 올려놓기 - `ReadIso14443aUidAsync` 대신

`ReadIso14443bUidAsync` 또는

`ReadIso15693UidAsync` 시도 (카드 타입에 따라 다름) - 01-ReadAnyUid 예제 로 모든 타입을 순차 시도

✖ .NET 8 예제를 .NET Framework 환경에서 빌드하면 오류 발생

원인: `NET8-Samples` 폴더의 예제는 `net8.0` 대상입니다.

해결: `NET4x-Samples` 폴더의 해당 예제를 사용합니다.

.NET 8 예제	대응하는 .NET 4.x 예제
<code>NET8-Samples/01-ReadAnyUid</code>	<code>NET4x-Samples/01-ReadAnyUid</code>
<code>NET8-Samples/03-MifareClassic</code>	<code>NET4x-Samples/03-MifareClassic</code>
<code>NET8-Samples/06-AutoRead</code>	<code>NET4x-Samples/06-AutoRead</code>
(동일한 번호로 대응)	...

✖ `ImplicitUsings` 관련 빌드 오류 (.NET Framework)

원인: .NET Framework 4.x 프로젝트에서 `ImplicitUsings=enable` 이 설정된 경우.

해결: `.csproj` 에서 비활성화합니다.

```
<ImplicitUsings>disable</ImplicitUsings>
```

이후 파일 상단에 필요한 `using` 을 모두 명시합니다.

```
using System;
using System.Threading;
using System.Threading.Tasks;
using Iksung.Reader;
using Iksung.Reader.Exceptions;
```

관련 문서

문서	내용
01. 시작하기	SDK 설치, 연결, Hello World
.NET 4.x 가이드	WinForms/WPF 통합, 문법 차이
API 레퍼런스	전체 공개 메서드 목록
13. 예외 처리	예외 종류 및 처리 패턴

© 익성전자 (Iksung Electronics). MIT License.

01. 시작하기 (Getting Started)

1. 패키지 설치

NuGet 패키지 관리자

```
Install-Package Iksung.Reader
```

.NET CLI

```
dotnet add package Iksung.Reader
```

프로젝트 파일 (.csproj)

```
<ItemGroup>
  <PackageReference Include="Iksung.Reader" Version="0.*" />
</ItemGroup>
```

2. 리더기 연결

Serial (UART) 연결

```
using Iksung.Reader;

await using var reader = await IksungReader.ConnectSerialAsync("COM3");
```

기본 파라미터:

파라미터	기본값	설명
portName	(필수)	포트 이름 (Windows: "COM3" , Linux: "/dev/ttyUSB0")
baudRate	115200	통신 속도
ct	default	취소 토큰

포트 이름 찾기

Windows — 장치 관리자 > 포트(COM 및 LPT) 에서 확인

Linux/macOS — `ls /dev/tty*` 또는 `ls /dev/cu.*`

3. 연결 확인

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

// 펌웨어 버전 출력
string version = await reader.ReadVersionAsync();
Console.WriteLine($"Firmware: {version}");

// 연결 상태 확인
Console.WriteLine($"Connected: {reader.IsConnected}");
Console.WriteLine($"Channel: {reader.ConnectedVia}");
```

4. 리소스 해제

`IksungReader` 는 `IAsyncDisposable` 을 구현합니다. `await using` 구문을 사용하면 블록 종료 시 자동으로 연결이 닫힙니다.

```
// 권장: await using 구문
await using var reader = await IksungReader.ConnectSerialAsync("COM3");
// ... 작업 수행 ...
// 블록 종료 시 자동 DisposeAsync() 호출

// 수동 해제가 필요한 경우
var reader = await IksungReader.ConnectSerialAsync("COM3");
try
{
    // ... 작업 수행 ...
}
finally
{
    await reader.DisposeAsync();
}
```

5. Hello World 예제

```

using Iksung.Reader;
using Iksung.Reader.Exceptions;

Console.WriteLine("[IKSUNG] Connecting...");
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

string fw = await reader.ReadVersionAsync();
Console.WriteLine($"[IKSUNG] Firmware: {fw}");

Console.WriteLine("[IKSUNG] Place a card on the reader...");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] uid = await reader.ReadIso14443aUidAsync(500, cts.Token);
        Console.WriteLine($"Card UID: {BitConverter.ToString(uid).Replace("-", "")}");
        await Task.Delay(500, cts.Token);
    }
    catch (IksungTimeoutException) { /* 카드 없음 - 계속 대기 */ }
    catch (OperationCanceledException) { break; }
}

Console.WriteLine("[IKSUNG] Done.");

```

6. 프로젝트 설정 권장사항

Top-level statements + `await using` 패턴

```

// Program.cs
using Iksung.Reader;

string port = args.Length > 0 ? args[0] : "COM3";
await using var reader = await IksungReader.ConnectSerialAsync(port);
// ...

```

CancellationToken 적용 (장시간 루프)


```
using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    // ... 반복 작업 ...
    try { await Task.Delay(200, cts.Token); }
    catch (OperationCanceledException) { break; }
}
```

예외 처리 템플릿

```
try
{
    byte[] uid = await reader.ReadIso14443aUidAsync();
    // ... 처리 ...
}
catch (IksungProtocolException ex)
{
    Console.WriteLine($"프로토콜 오류: {ex.Message}");
}
catch (IksungTimeoutException)
{
    // 카드 없음 또는 응답 없음
}
```

자세한 예외 처리는 [13. 예외 처리 및 고급 사용](#)을 참조하세요.

02. 공통 명령 (Common Commands)

리더기 자체 정보를 읽고 RF 출력을 제어하는 기본 명령들입니다.

1. 펌웨어 버전 읽기

```
string version = await reader.ReadVersionAsync();
Console.WriteLine($"Firmware: {version}");
// 출력 예: "Firmware: IS-3500K V1.8.0"
```

파라미터	기본값	설명
timeoutMs	1000	응답 대기 최대 시간 (ms)
ct	default	취소 토큰

2. 리더기 고유 ID 읽기

리더기 하드웨어에 고유하게 부여된 식별자(시리얼 번호)를 읽습니다.

```
byte[] uid = await reader.ReadUniqueIdAsync();
string uidHex = BitConverter.ToString(uid).Replace("-", "");
Console.WriteLine($"Reader UID: {uidHex}");
```

3. RF 안테나 제어

RF 켜기

```
await reader.RfOnAsync();
```

RF 끄기

```
await reader.RfOffAsync();
```

주의: RF를 끄면 카드와의 통신이 즉시 중단됩니다. 연속 읽기 루프 사이에 RF를 켜다 켜면 카드 중복 감지를 방지할 수 있습니다.

4. 빠른 UID 읽기 (카드 타입별)

상세 활성화 없이 UID만 빠르게 읽을 때 사용합니다.

```
// ISO 14443 Type A (Mifare 계열 포함)
byte[] uidA = await reader.ReadIso14443aUidAsync();

// ISO 14443 Type B
byte[] uidB = await reader.ReadIso14443bUidAsync();

// ISO 15693 (ICODE 계열 등 HF 스티커 태그)
byte[] uid15 = await reader.ReadIso15693UidAsync();

// LF 125 kHz (EM410X 등)
byte[] uidLf = await reader.ReadLf125KhzUidAsync();
```

카드가 없으면 `IksungTimeoutException` 이 발생합니다.

5. 전형적인 폴링 루프

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] uid = await reader.ReadIso14443aUidAsync(500, cts.Token);
        Console.WriteLine($"{DateTime.Now:HH:mm:ss} UID: {BitConverter.ToString(uid).Replace("-", "")}");
        await Task.Delay(300, cts.Token); // 중복 감지 방지 딜레이
    }
    catch (IksungTimeoutException) { /* 카드 없음 */ }
    catch (OperationCanceledException) { break; }
}
```

6. 부저 울리기

내장 부저를 짧게 울려 카드 인식 피드백을 제공합니다. `durationUnit` 은 100 ms 단위입니다 (기본값 2 = 약 200 ms).

```
await reader.BuzzerAsync(); // 기본 약 200 ms
await reader.BuzzerAsync(5); // 약 500 ms
```

파라미터	기본값	설명
<code>durationUnit</code>	2	울림 시간 단위 (1 = 100 ms, 2 = 200 ms, ...)
<code>timeoutMs</code>	1000	응답 대기 시간

7. 자동 카드 감지 (모든 표준)

카드 종류에 관계없이 현재 RF 필드의 카드 UID 와 종류를 한 번에 얻습니다. ISO 14443 A/B, FeliCa, ISO 15693 등 표준이 섞여 있는 환경에서 유용합니다.

```
byte[] uid = await reader.ReadAllUidAsync(); // UID 만
byte[] typeCode = await reader.ReadAllCardTypeAsync(); // 카드 종류 코드

Console.WriteLine($"UID: {BitConverter.ToString(uid).Replace("-", "")}");
Console.WriteLine($"카드 종류 코드: {BitConverter.ToString(typeCode)}");
```

카드 종류별 폴링을 명시적으로 하고 싶다면 `ReadIso14443aUidAsync` / `ReadIso14443bUidAsync` / `ReadFelicaUidAsync` / `ReadIso15693UidAsync` 를 직접 호출하세요.

FeliCa 전용 UID

```
byte[] uidFelica = await reader.ReadFelicaUidAsync();
```

8. 카드 메타 정보 (활성화된 카드)

ISO 14443-A/B 카드가 이미 활성화된 상태에서 추가 메타데이터를 조회합니다. `Iso14443ActivateAsync` 등으로 카드를 활성화한 직후 호출합니다 (§ 03 참조).

```
byte[] sak      = await reader.ReadSakTypeAsync();    // SAK (Select Acknowledge) - 1 byte
byte[] ats      = await reader.ReadAtsAsync();        // ATS (Answer To Select) - ISO 14443
byte[] tagType  = await reader.ReadTagTypeAsync();    // 펌웨어가 분류한 태그 타입 코드
```

메서드	용도
<code>ReadSakTypeAsync</code>	Mifare Classic/Plus/DESFire 구분 (예: <code>0x08</code> = Classic 1K)
<code>ReadAtsAsync</code>	Layer-4 카드의 ATS / HistoricalBytes 확인
<code>ReadTagTypeAsync</code>	펌웨어 종합 분류 (Mifare 패밀리 / NTAG / DESFire 등)

9. 교통카드 일련번호

국내 교통카드(T-Money / CashBee / K-Cash / RailPlus)의 일련번호를 읽습니다. 카드 종류를 알 수 없을 때는 `ReadAllCashSerialAsync` 로 자동 감지합니다.

```
byte[] tmoney    = await reader.ReadTmoneySerialAsync();
byte[] cashbee   = await reader.ReadCashbeeSerialAsync();
byte[] kcash     = await reader.ReadKcashSerialAsync();
byte[] railplus  = await reader.ReadRailPlusSerialAsync();

// 4 종 자동 감지 - 어떤 교통카드든 단일 호출로 일련번호 획득
byte[] anyCash   = await reader.ReadAllCashSerialAsync();
```

일련번호가 BCD 또는 ASCII 인코딩이라 카드사별로 표시 방식이 다릅니다. 결제 시스템과 연동하기 전에 카드사 사양서를 함께 확인하세요.

10. Watchdog Timer (WDT) 설정

리더기가 일정 시간 동안 정상 동작하지 않으면 내부 워치독으로 자동 리셋합니다. 타임아웃 값은 펌웨어 정의 단위입니다 (제품 매뉴얼 참조).

```
// 현재 설정값 읽기
byte[] current = await reader.ReadWdtTimeoutAsync();
Console.WriteLine($"WDT timeout: {current[0]}");

// 새 값 설정 (예: 10 단위)
await reader.WriteWdtTimeoutAsync(10);
```

메서드	용도
<code>ReadWdtTimeoutAsync</code>	현재 WDT 타임아웃 값 조회
<code>WriteWdtTimeoutAsync(byte timeoutValue)</code>	WDT 타임아웃 값 설정 (펌웨어 정의 단위)

03. ISO 14443 A/B — 비접촉 카드

개요

ISO 14443는 13.56 MHz 비접촉 카드 표준입니다.

타입	대표 카드	설명
Type A	Mifare Classic, Mifare Ultralight, NTag, DESFire	가장 흔한 비접촉 카드 계열
Type B	주민등록증, 여권 IC칩, 일부 교통카드	T=0/T=1 프로토콜 사용
Type A+B	두 타입 동시 감지	다중 카드 환경

1. Layer-3 활성화 (UID 획득)

Layer-3 활성화는 카드의 UID와 기본 정보를 반환합니다.

```
// Type A 활성화 (Mifare 계열)
byte[] atr = await reader.ActivateIso14443aAsync();
Console.WriteLine($"ATR(A): {BitConverter.ToString(atr).Replace("-", " ")");

// Type B 활성화
byte[] atrB = await reader.ActivateIso14443bAsync();

// Type A와 B 동시 시도 (먼저 응답하는 타입 반환)
byte[] atrAB = await reader.ActivateIso14443abAsync();
```

2. Layer-4 활성화 (ISO-DEP / APDU 교환 준비)

EMV 결제 카드, 스마트카드처럼 APDU 명령을 사용하는 카드에는 Layer-4 활성화가 필요합니다.

```
// ISO-DEP Layer-4 활성화 (T=CL 프로토콜 수립)
byte[] atr4 = await reader.ActivateIso14443_4aAsync();

// Layer-3 + Layer-4 동시 활성화
byte[] atr34 = await reader.ActivateIso14443_3a4aAsync();
```

3. APDU 교환

Layer-4 활성화 후 ISO 7816-4 APDU 명령을 보낼 수 있습니다.

```
byte[] atr = await reader.ActivateIso14443_4aAsync();

// SELECT PPSE (EMV 결제 단말 환경 선택)
byte[] selectPpse = [
    0x00, 0xA4, 0x04, 0x00, 0x0E,
    0x32, 0x50, 0x41, 0x59, 0x2E, 0x53, 0x59, 0x53,
    0x2E, 0x44, 0x44, 0x46, 0x30, 0x31,
    0x00
];
byte[] response = await reader.ExchangeApuAsync(selectPpse, 3000);

// SW 코드 파싱
byte sw1 = response[^2];
byte sw2 = response[^1];
Console.WriteLine($"SW: {sw1:X2}{sw2:X2}");

if (sw1 == 0x90 && sw2 == 0x00)
{
    byte[] data = response[..^2]; // SW 제외한 데이터
    Console.WriteLine($"Response data: {BitConverter.ToString(data).Replace("-", " ")}");
}
```

자주 쓰는 SW 코드:

SW	의미
9000	성공
6A80	잘못된 데이터
6A82	파일 또는 앱 없음
6300	인증 실패 (카운터 포함)
6700	잘못된 길이
6E00	CLA 미지원

4. 카드 비활성화 (Halt)

카드 읽기가 끝나면 Halt 명령으로 카드를 비활성화합니다.


```
await reader.HaltIso14443aAsync();
await reader.HaltIso14443bAsync();
```

Halt 후 RF Off → RF On 사이클 없이도 새 카드를 인식할 수 있습니다.

5. 전체 예제 — SELECT PPSE

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] atr = await reader.ActivateIso14443_4aAsync(1000, cts.Token);
        Console.WriteLine($"Card activated. ATR: {BitConverter.ToString(atr).Replace("-", "")}");

        byte[] selectPpse = [
            0x00, 0xA4, 0x04, 0x00, 0x0E,
            0x32, 0x50, 0x41, 0x59, 0x2E, 0x53, 0x59, 0x53,
            0x2E, 0x44, 0x44, 0x46, 0x30, 0x31, 0x00
        ];
        byte[] resp = await reader.ExchangeApuAsync(selectPpse, 3000, cts.Token);

        string sw = $"{resp[^2]:X2}{resp[^1]:X2}";
        Console.WriteLine($"PPSE Response SW: {sw}");

        await reader.HaltIso14443aAsync(500, cts.Token);
        await Task.Delay(500, cts.Token);
    }
    catch (IksungTimeoutException) { await Task.Delay(200, cts.Token); }
    catch (OperationCanceledException) { break; }
}
```

04. Mifare Classic

개요

Mifare Classic은 NXP의 ISO 14443 Type A 기반 칩입니다. 출입 통제, 교통, 사원증 등에 널리 사용됩니다.

제품	용량	섹터	블록
Mifare Classic 1K	1 KB	16 섹터	64 블록 (블록 0~63)
Mifare Classic 4K	4 KB	40 섹터	256 블록

메모리 구조 (1K 기준)

```
섹터 0 : 블록 0(Manufacturer), 블록 1, 블록 2, 블록 3(Sector Trailer)
섹터 1 : 블록 4, 블록 5, 블록 6, 블록 7(Sector Trailer)
...
섹터 15: 블록 60, 블록 61, 블록 62, 블록 63(Sector Trailer)
```

- **블록 0:** 제조사 데이터 (읽기 전용, 쓰기 불가)
- **Sector Trailer (각 섹터 마지막 블록):** Key A (6B), Access Bits (4B), Key B (6B)

1. 카드 활성화

```
byte[] atr = await reader.ActivateMifareAsync();
Console.WriteLine($"Mifare activated: {BitConverter.ToString(atr).Replace("-", " ") }");
```

2. 인증

블록을 읽거나 쓰기 전에 해당 섹터의 Key로 인증해야 합니다.

```
using Iksung.Reader.Models; // MifareKeyType

byte blockNo = 4; // 섹터 1의 첫 번째 블록

// Key A로 인증 (기본 공장 키)
byte[] keyA = [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF];
await reader.MifareAuthenticateAsync(blockNo, MifareKeyType.KeyA, keyA);

// Key B로 인증
byte[] keyB = [0xB0, 0xB1, 0xB2, 0xB3, 0xB4, 0xB5];
await reader.MifareAuthenticateAsync(blockNo, MifareKeyType.KeyB, keyB);
```

같은 섹터 내에서는 한 번 인증하면 해당 섹터의 모든 블록에 접근 가능합니다.

3. 블록 읽기

```
byte[] data = await reader.MifareReadBlockAsync(blockNo);
Console.WriteLine($"Block {blockNo}: {BitConverter.ToString(data).Replace("-", " ") }");
// 출력 예: "00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF"
// (항상 16바이트 반환)
```

4. 블록 쓰기

```
byte[] newData = new byte[16]; // 반드시 16바이트
Array.Fill(newData, (byte)0x00);
newData[0] = 0xAB;
newData[1] = 0xCD;

await reader.MifareWriteBlockAsync(blockNo, newData);
Console.WriteLine("Block written.");
```

주의: 블록 0 (제조사 블록)과 Sector Trailer에 잘못된 데이터를 쓰면 카드가 영구 손상될 수 있습니다.

5. 섹터 읽기

섹터의 데이터 블록 전체를 한 번에 읽습니다 (Sector Trailer 제외).

```
byte sectorNo = 1;
byte[] sectorData = await reader.MifareReadSectorAsync(sectorNo);
// sectorData = 섹터 내 데이터 블록들의 합산 (각 블록 16바이트)
```

6. 전체 예제 — 블록 읽기/쓰기

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

Console.WriteLine("Mifare Classic 카드를 올려 주세요...");

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] atr = await reader.ActivateMifareAsync(1000, cts.Token);
        Console.WriteLine($"카드 활성화: {BitConverter.ToString(atr).Replace("-", " ")}");

        byte blockNo = 4;
        byte[] key = [0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF];

        // 인증
        await reader.MifareAuthenticateAsync(blockNo, MifareKeyType.KeyA, key, 1000, cts);
        Console.WriteLine("인증 성공");

        // 읽기
        byte[] before = await reader.MifareReadBlockAsync(blockNo, 1000, cts.Token);
        Console.WriteLine($"읽기 전: {BitConverter.ToString(before).Replace("-", " ")}");

        // 현재 시각을 블록에 기록
        byte[] write = new byte[16];
        BitConverter.GetBytes(DateTimeOffset.UtcNow.ToUnixTimeSeconds()).CopyTo(write, 0);
        await reader.MifareWriteBlockAsync(blockNo, write, 1000, cts.Token);
        Console.WriteLine("쓰기 완료");

        // 검증
        byte[] after = await reader.MifareReadBlockAsync(blockNo, 1000, cts.Token);
        Console.WriteLine($"읽기 후: {BitConverter.ToString(after).Replace("-", " ")}");

        await Task.Delay(2000, cts.Token);
    }
    catch (IksungTimeoutException) { await Task.Delay(200, cts.Token); }
    catch (OperationCanceledException) { break; }
    catch (Exception ex) { Console.WriteLine($"오류: {ex.Message}"); }
}
```

7. 섹터 번호 ↔ 블록 번호 변환 (1K)

```
// 섹터 번호로 데이터 블록 번호 계산
static byte SectorToBlock(byte sector, byte blockInSector = 0)
    => (byte)(sector * 4 + blockInSector);

// 예
byte dataBlock = SectorToBlock(2, 0); // = 8 (섹터 2의 첫 번째 블록)
byte trailerBlock = SectorToBlock(2, 3); // = 11 (섹터 2의 Sector Trailer)
```

05. Mifare Ultralight / NTag

개요

Mifare Ultralight와 NTag는 NXP의 저비용 ISO 14443 Type A 메모리 태그입니다. 스티커, 포스터, 소모품 추적 등에 주로 사용됩니다.

제품	페이지 수	사용자 메모리	특징
Mifare Ultralight	16	48 바이트	기본형, 암호화 없음
Mifare Ultralight C	48	144 바이트	3DES 인증 지원
NTag 213	45	144 바이트	비밀번호 보호, ECC 서명
NTag 215	135	504 바이트	대용량
NTag 216	231	888 바이트	최대 용량

각 페이지는 **4바이트**입니다.

1. 카드 활성화

```
byte[] atr = await reader.ActivateMifareUltralightAsync();
Console.WriteLine($"UL activated: {BitConverter.ToString(atr).Replace("-", " ") }");
```

2. 페이지 읽기

```
byte page = 4; // 사용자 메모리 시작 페이지 (NTag213 기준)
byte[] data = await reader.MifareUltralightReadPageAsync(page);
// 리더기는 지정 페이지부터 4페이지(16바이트)를 반환하는 경우도 있음
Console.WriteLine($"Page {page}: {BitConverter.ToString(data).Replace("-", " ") }");
```

NTag213 메모리 레이아웃:

페이지	내용
0~1	UID (7바이트) + BCC
2	UID(6~7) + Internal
3	OTP (One-Time Programmable)
4~39	사용자 데이터 (144바이트)
40	CFG 0 (AUTH0 등 설정)
41	CFG 1 (Access 설정)
42	PWD (비밀번호 — 읽기 불가)
43	PACK (비밀번호 응답)
44	내부

3. 페이지 쓰기

```
byte page = 5;
byte[] data = [0x12, 0x34, 0x56, 0x78]; // 반드시 4바이트
await reader.MifareUltralightWritePageAsync(page, data);
Console.WriteLine("Written.");
```

주의: 페이지 0~3은 제조사 / OTP 영역이므로 쓰지 마세요.

4. NTag 전용 기능

버전 정보 읽기

```
byte[] ver = await reader.NTagGetVersionAsync();
string chipType = IksungReader.ParseNTagType(ver);
Console.WriteLine($"Chip type: {chipType}");
// 출력 예: "NTag213"
```

빠른 읽기 (FastRead)

여러 페이지를 한 번에 읽습니다.

```
byte startPage = 4;
byte endPage   = 15;
byte[] pages = await reader.NTagFastReadAsync(startPage, endPage);
// pages.Length = (endPage - startPage + 1) * 4
```

NFC 카운터 읽기

NTag215/216은 태그가 리더기에 근접할 때마다 자동으로 증가하는 카운터를 제공합니다.

```
byte[] counter = await reader.NTagReadCounterAsync(0); // counterNo=0
int count = (counter[0]) | (counter[1] << 8) | (counter[2] << 16);
Console.WriteLine($"NFC Counter: {count}");
```

ECC 서명 읽기

NXP 공장에서 주입된 32바이트 ECC 서명을 읽어 칩의 진위를 확인합니다.

```
byte[] sig = await reader.NTagReadSignatureAsync();
Console.WriteLine($"ECC Signature: {BitConverter.ToString(sig).Replace("-", " ")}");
// 32바이트 서명
```

5. 비밀번호 보호

비밀번호 인증

비밀번호 보호가 설정된 태그에 접근 시 먼저 인증해야 합니다.

```
byte[] password = [0x01, 0x02, 0x03, 0x04]; // 4바이트 비밀번호
await reader.NTagPasswordAuthAsync(password);
Console.WriteLine("비밀번호 인증 성공");
```

AUTH0 설정 (보호 시작 페이지)

AUTH0 페이지 번호 이상의 페이지에 비밀번호 인증이 요구되도록 설정합니다.

```
byte auth0 = 4; // 페이지 4부터 비밀번호 보호
await reader.NTagWriteAuth0Async(auth0);
```

`auth0 = 0xFF` 로 설정하면 비밀번호 보호가 비활성화됩니다.

Access 설정

읽기 보호 여부, 쓰기 보호 여부를 설정합니다.

```
// bit7=PROT(1=읽기도 보호), bit6=CFGLCK, bit2:0=NFC_CNT_PKG  
byte access = 0x00; // 쓰기만 보호 (읽기는 자유)  
await reader.NTagWriteAccessAsync(access);
```

비밀번호 변경

```
byte[] newPwd = [0xAA, 0xBB, 0xCC, 0xDD]; // 새 비밀번호 4바이트  
byte[] newPack = [0x12, 0x34]; // 응답 코드 2바이트  
await reader.NTagChangePasswordAsync(newPwd, newPack);  
Console.WriteLine("비밀번호 변경 완료");
```

비밀번호를 잊으면 해당 영역에 접근할 수 없으므로 신중하게 설정하세요.

6. 전체 예제 — NTag213 전체 덤프

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

while (true)
{
    try
    {
        byte[] atr = await reader.ActivateMifareUltralightAsync();
        Console.WriteLine($"Tag: {BitConverter.ToString(atr).Replace("-", " ")");

        // 버전으로 칩 타입 확인
        byte[] ver = await reader.NTagGetVersionAsync();
        Console.WriteLine($"Chip: {IksungReader.ParseNTagType(ver)}");

        // 페이지 0~44 전체 읽기
        byte[] all = await reader.NTagFastReadAsync(0, 44);
        for (int i = 0; i < all.Length / 4; i++)
        {
            byte[] page = all[(i * 4)..((i + 1) * 4)];
            Console.WriteLine($"  Page {i,2}: {BitConverter.ToString(page).Replace("-", " ")");
        }

        await Task.Delay(3000);
    }
    catch (IksungTimeoutException) { await Task.Delay(200); }
}
```

06. ISO 15693 — HF 장거리 태그

개요

ISO 15693은 13.56 MHz HF 대역에서 동작하는 비접촉 태그 표준입니다.

ISO 14443 대비 인식 거리가 길어 재고 관리, 도서관 도서 태그, 물류 스티커에 많이 사용됩니다.

대표 제품	제조사	특징
ICODE SLI	NXP	가장 흔한 ISO 15693 태그
ICODE SLIX	NXP	EAS 잠금, 개인정보 보호
Tag-it HF-I	Texas Instruments	저전력
ST25TV	STMicroelectronics	NFC + ISO15693 복합

1. 카드 활성화

```
byte[] atr = await reader.ActivateIso15693Async();
// atr: UID 8바이트 (리를 엔디언 순서로 반환됨)
Console.WriteLine($"UID (LE): {BitConverter.ToString(atr).Replace("-", " ")}");

// 사람이 읽기 좋은 형태로 역순 표시
string uidDisplay = string.Join(":", atr.Reverse().Select(b => $"{b:X2}"));
Console.WriteLine($"UID      : {uidDisplay}");
```

2. 단일 블록 읽기

```
byte blockNo = 0;
byte[] data = await reader.Iso15693ReadBlockAsync(blockNo);
Console.WriteLine($"Block {blockNo}: {BitConverter.ToString(data).Replace("-", " ")}");
// 블록 크기는 태그마다 다름 (보통 4바이트 또는 8바이트)
```

3. 멀티 블록 읽기

여러 블록을 한 번의 명령으로 읽어 성능을 향상시킵니다.

```
byte firstBlock = 0;
byte blockCount = 8; // 0번부터 7번까지 8개 블록

byte[] data = await reader.Iso15693ReadMultipleBlocksAsync(firstBlock, blockCount, 2000);

int blockSize = data.Length / blockCount;
for (int i = 0; i < blockCount; i++)
{
    byte[] block = data[(i * blockSize)..((i + 1) * blockSize)];
    Console.WriteLine($"Block {firstBlock + i,2}: {BitConverter.ToString(block).Replace("0", " ")}");
}
```

4. 단일 블록 쓰기

```
byte blockNo = 1;
byte[] data = [0xDE, 0xAD, 0xBE, 0xEF]; // 태그 블록 크기에 맞춰야 함
await reader.Iso15693WriteBlockAsync(blockNo, data);
Console.WriteLine("Written.");
```

쓰기 잠금(Write-locked)된 블록에 쓰면 `IksungProtocolException` 이 발생합니다.

5. 전체 예제 — 블록 전체 덤프

```

await using var reader = await IksungReader.ConnectSerialAsync("COM3");
Console.WriteLine("ISO 15693 태그를 올려 주세요...\n");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] uid = await reader.ActivateIso15693Async(1000, cts.Token);
        string uidStr = string.Join(":", uid.Reverse().Select(b => $"{b:X2}"));
        Console.WriteLine($"UID: {uidStr}");

        // 멀티 블록 읽기 시도
        try
        {
            byte[] all = await reader.Iso15693ReadMultipleBlocksAsync(0, 8, 2000, cts.Token);
            int blockSize = all.Length / 8;
            for (int i = 0; i < 8; i++)
            {
                byte[] blk = all[(i * blockSize)..((i + 1) * blockSize)];
                Console.WriteLine($"  Block {i,2}: {BitConverter.ToString(blk).Replace("-", " ")}");
            }
        }
        catch (IksungProtocolException)
        {
            // 멀티블록 미지원 시 단일 블록으로 폴백
            for (byte b = 0; b < 8; b++)
            {
                try
                {
                    byte[] blk = await reader.Iso15693ReadBlockAsync(b, 1000, cts.Token);
                    Console.WriteLine($"  Block {b,2}: {BitConverter.ToString(blk).Replace("-", " ")}");
                }
                catch { Console.WriteLine($"  Block {b,2}: (읽기 실패)"); }
            }
        }

        Console.WriteLine();
        await Task.Delay(3000, cts.Token);
    }
    catch (IksungTimeoutException) { await Task.Delay(200, cts.Token); }
    catch (OperationCanceledException) { break; }
}

```

07. DESFire EV1 / EV2 / EV3

개요

Mifare DESFire는 NXP의 고보안 ISO 14443 Type A 스마트카드입니다.

파일 시스템 구조와 AES/DES 암호화를 내장하여 전자화폐, 출입 통제, 대중교통에 사용됩니다.

메모리 구조

```
PICC (카드)
├─ Root Application (AID = 0000000)
│   └─ Application (AID = 예: 010203)
│       ├── File 01 (Standard Data File)
│       ├── File 02 (Value File)
│       └─ File 03 (Record File)
```

암호화 방식 (cryptoType)

값	알고리즘	키 길이
0x00	DES / 2TDEA	8 / 16 바이트
0x01	3TDEA (3K3DES)	24 바이트
0x02	AES-128	16 바이트

1. 카드 활성화

```
byte[] atr = await reader.ActivateDesfireAsync();
Console.WriteLine($"DESFire ATR: {BitConverter.ToString(atr).Replace("-", " ") }");
```

2. 카드 정보 조회

```
// 남은 메모리 (바이트)
byte[] freeMemRaw = await reader.DesfireGetFreeMemoryAsync();
int freeMem = (freeMemRaw[0]) | (freeMemRaw[1] << 8) | (freeMemRaw[2] << 16);
Console.WriteLine($"Free memory: {freeMem} bytes");

// 랜덤 UID 읽기 (RandomUID 모드일 때)
byte[] uid = await reader.DesfireGetUidAsync();
Console.WriteLine($"UID: {BitConverter.ToString(uid).Replace("-", " ") }");
```

3. 키 관리

DESFire 인증에 사용할 키를 리더기의 플래시 메모리에 저장합니다.
(키는 리더기 내부에서만 사용되며 외부로 노출되지 않습니다.)

```
byte keyIndex = 0; // 리더기 내부 키 슬롯 번호 (0~N)
byte keyVersion = 0x00; // 키 버전
byte cryptoType = 0x02; // 0x02 = AES-128

byte[] aesKey = [
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
];

await reader.DesfireKeySaveAsync(keyIndex, keyVersion, cryptoType, aesKey);
Console.WriteLine("Key saved to reader flash.");
```

키 초기화 (공장 기본값으로 리셋)

```
await reader.DesfireKeyInitAsync();
```

4. 인증

저장된 키를 사용하여 카드와 상호 인증합니다.

```
// AES 인증 (EV1/EV2/EV3)
byte keyNo = 0x00; // 카드의 키 번호 (보통 마스터 키 = 0x00)
await reader.DesfireAuthenticateAesAsync(keyNo);
Console.WriteLine("AES 인증 성공");

// 2K3DES 인증 (구형 EV1)
await reader.DesfireAuthenticate2K3DesAsync(keyNo);

// ISO 인증 (EV2/EV3 고보안 모드)
await reader.DesfireAuthenticateIsoAsync(keyNo);

// 인증 초기화 (인증 상태 취소)
await reader.DesfireAuthResetAsync();
```

5. 애플리케이션 관리

애플리케이션 목록 조회

```
byte[] appIdsRaw = await reader.DesfireGetApplicationIdsAsync();
// 3바이트씩 AID 목록
for (int i = 0; i + 2 < appIdsRaw.Length; i += 3)
{
    byte[] aid = appIdsRaw[i..(i + 3)];
    Console.WriteLine($"AID: {BitConverter.ToString(aid).Replace("-", " ")}");
}
```

애플리케이션 선택

```
byte[] appId = [0x01, 0x02, 0x03];
await reader.DesfireSelectApplicationAsync(appId);
Console.WriteLine($"Application {BitConverter.ToString(appId).Replace("-", " ")} selected.

// 루트(PICC) 선택
await reader.DesfireSelectRootAsync();
```

애플리케이션 생성

```
byte[] newAppId = [0x01, 0x02, 0x03];
byte keySettings = 0x0F; // 키 설정 (상세 내용은 DESFire 데이터시트 참조)
byte keyCount = 0x01; // 이 앱에 사용할 키 개수

await reader.DesfireCreateApplicationAsync(newAppId, keySettings, keyCount);
Console.WriteLine("Application created.");
```


애플리케이션 삭제

```
byte[] appId = [0x01, 0x02, 0x03];  
await reader.DesfireDeleteApplicationAsync(appId);
```

6. 파일 관리

파일 목록 조회

```
byte[] fileIds = await reader.DesfireGetFileIdsAsync();  
Console.WriteLine($"Files: {BitConverter.ToString(fileIds).Replace("-", " ") }");
```

파일 설정 조회

```
byte fileNo = 0x01;  
byte[] settings = await reader.DesfireGetFileSettingsAsync(fileNo);  
Console.WriteLine($"File {fileNo} settings: {BitConverter.ToString(settings).Replace("-",
```

Standard Data File 생성

```
byte fileNo      = 0x01;  
byte commSettings = 0x00;    // 0x00=평문, 0x01=MAC, 0x03=암호화  
ushort accessRights = 0x0000; // 키 번호 조합 (상세 내용은 데이터시트 참조)  
int fileSize      = 32;      // 파일 크기 (바이트)  
  
await reader.DesfireCreateStdFileAsync(fileNo, commSettings, accessRights, fileSize);  
Console.WriteLine("Standard file created.");
```

파일 삭제

```
await reader.DesfireDeleteFileAsync(0x01);
```

7. 데이터 파일 읽기/쓰기

읽기

```
byte fileNo      = 0x01;
byte commSettings = 0x00; // 평균
int offset       = 0;
int length       = 0;      // 0 = 파일 전체

byte[] data = await reader.DesfireReadDataFileAsync(fileNo, commSettings, offset, length);
Console.WriteLine($"Data: {BitConverter.ToString(data).Replace("-", " ")");
```

쓰기 + 트랜잭션 커밋

```
byte[] payload = System.Text.Encoding.UTF8.GetBytes("Hello DESFire!");

await reader.DesfireWriteDataFileAsync(0x01, payload);
await reader.DesfireCommitTransactionAsync();
Console.WriteLine("Write committed.");

// 트랜잭션 취소
// await reader.DesfireAbortTransactionAsync();
```

8. 밸류 파일 (Value File)

포인트, 잔액처럼 정수값을 안전하게 저장합니다.

```
// 현재 값 읽기
byte[] raw = await reader.DesfireReadValueFileAsync(0x02);
int value = BitConverter.ToInt32(raw, 0);
Console.WriteLine($"Balance: {value}");

// 충전 (Credit)
await reader.DesfireCreditValueFileAsync(0x02, 1000);
await reader.DesfireCommitTransactionAsync();
Console.WriteLine("Credited 1000.");

// 차감 (Debit)
await reader.DesfireDebitValueFileAsync(0x02, 300);
await reader.DesfireCommitTransactionAsync();
Console.WriteLine("Debited 300.");
```

9. 카드 포맷 (공장 초기화)

주의: 모든 데이터와 앱이 삭제됩니다. PICC 마스터 키로 인증된 상태에서만 가능합니다.

```
await reader.DesfireFormatAsync(3000);  
Console.WriteLine("Card formatted.");
```

10. 전체 워크플로우 예제

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");  
  
// 1. 활성화  
byte[] atr = await reader.ActivateDesfireAsync();  
Console.WriteLine($"DESFire activated: {BitConverter.ToString(atr).Replace("-", " ")}");  
  
// 2. 키 저장  
byte[] aesKey = new byte[16]; // All-zero AES key (공장 기본값)  
await reader.DesfireKeySaveAsync(0, 0, 0x02, aesKey);  
  
// 3. 루트 선택 + 마스터 키 인증  
await reader.DesfireSelectRootAsync();  
await reader.DesfireAuthenticateAesAsync(0x00);  
Console.WriteLine("Root authenticated.");  
  
// 4. 앱 생성  
byte[] aid = [0xAA, 0xBB, 0xCC];  
await reader.DesfireCreateApplicationAsync(aid, 0x0F, 1);  
  
// 5. 앱 선택 + 인증  
await reader.DesfireSelectApplicationAsync(aid);  
await reader.DesfireAuthenticateAesAsync(0x00);  
  
// 6. 파일 생성  
await reader.DesfireCreateStdFileAsync(0x01, 0x00, 0x0000, 32);  
  
// 7. 쓰기  
byte[] data = new byte[32];  
System.Text.Encoding.ASCII.GetBytes("Hello DESFire!").CopyTo(data, 0);  
await reader.DesfireWriteDataFileAsync(0x01, data);  
await reader.DesfireCommitTransactionAsync();  
  
// 8. 읽기  
byte[] read = await reader.DesfireReadDataFileAsync(0x01);  
Console.WriteLine($"Read: {System.Text.Encoding.ASCII.GetString(read).TrimEnd('\0')}");
```

08. LF 125 kHz

개요

LF(Low Frequency) 125 kHz 태그는 출입 통제, 동물 칩, 주차 시스템에 널리 쓰이는 오래된 RF 표준입니다.

포맷	용도
EM410X	공장/사무소 출입 카드 (가장 보편적)
ISO 11784/11785 FDX-B	동물 ID 마이크로칩
SECOM	국내 보안 시스템 전용
Temic (T5577)	재기록 가능한 범용 LF 칩
Raw bits	알 수 없는 포맷 디버그 용도

1. 기본 UID 읽기 (EM410X)

```
byte[] uid = await reader.ReadLf125KhzUidAsync();
Console.WriteLine($"EM410X UID: {BitConverter.ToString(uid).Replace("-", " ")}");
// 출력 예: "1234567890" (5바이트 = 10자리 16진수)
```

```
// Raw UID (원시 바이트)
byte[] rawUid = await reader.ReadLf125KhzRawUidAsync();
Console.WriteLine($"Raw UID: {BitConverter.ToString(rawUid).Replace("-", " ")}");
```

2. ISO 11784/11785 — 동물 ID 칩

FDX-B 포맷으로 인코딩된 동물 ID 칩을 읽습니다.

```
byte[] raw = await reader.ReadLfIso11784Async();
Console.WriteLine($"FDX-B raw: {BitConverter.ToString(raw).Replace("-", " ")");

// 64비트 파싱
ulong bits = 0;
for (int i = 0; i < 8; i++) bits |= ((ulong)raw[i]) << (i * 8);

ulong animalId = bits & 0x3FFFFFFF; // 하위 38비트: 개체 번호
ulong countryId = (bits >> 38) & 0xFF; // 상위 10비트: 국가 코드

Console.WriteLine($"국가 코드: {countryId:D3}");
Console.WriteLine($"개체 번호: {animalId:D12}");
// 국가 코드 410 = 대한민국
```

Raw Low Data (저수준)

```
byte[] lowData = await reader.ReadLfIso11784LowDataAsync();
```

3. SECOM 블록 읽기

국내 보안 시스템에서 사용하는 SECOM 포맷 블록을 읽습니다.

```
byte[] secom = await reader.ReadLfSecomBlockAsync();
Console.WriteLine($"SECOM: {BitConverter.ToString(secom).Replace("-", " ")");
```

4. Temic 블록 읽기

T5577 범용 재기록 가능 칩의 데이터를 읽습니다.

```
byte readBits = 0x40; // 읽을 비트 수 (64비트)
byte delayMs = 0x0A; // 딜레이 (10ms)

byte[] temic = await reader.ReadLfTemicBlockAsync(readBits, delayMs);
Console.WriteLine($"Temic: {BitConverter.ToString(temic).Replace("-", " ")");

// Raw Low Data
byte[] temicRaw = await reader.ReadLfTemicLowDataAsync();
```

5. Raw 비트 스트림 읽기

포맷을 알 수 없는 칩을 디버그할 때 원시 RF 비트를 읽습니다.

```
byte bitCount = 64;
byte[] bits = await reader.ReadLfRawBitsAsync(bitCount);

// 비트 문자열로 변환
var sb = new System.Text.StringBuilder();
for (int i = 0; i < bitCount && (i / 8) < bits.Length; i++)
{
    sb.Append((bits[i / 8] >> (7 - (i % 8))) & 1);
    if ((i + 1) % 8 == 0) sb.Append(' ');
}
Console.WriteLine($"Bits: {sb}");
```

6. HTRC 샘플링 자동 조정

LF 리더기의 수신 특성을 환경에 맞게 자동 최적화합니다.

첫 실행 시 또는 환경(온도, 금속 주변부 등)이 바뀔 경우에 한 번 실행하는 것을 권장합니다.

```
// 자동 튜닝 (최대 5초 소요)
byte[] result = await reader.AutoTuneLfSamplingAsync(5000);
Console.WriteLine($"Auto-tune result: 0x{result[0]:X2}");

// 현재 샘플링 시간 읽기
byte[] samplingTime = await reader.ReadLfHtrcSamplingTimeAsync();
Console.WriteLine($"Sampling time: {samplingTime[0]}");

// 타이밍 파라미터 읽기
byte[] timing = await reader.ReadLfTimingParamsAsync();
Console.WriteLine($"Timing params: {BitConverter.ToString(timing).Replace("-", " ")}");
```

7. 전체 예제 — 다중 포맷 자동 감지

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

// HTRC 자동 조정
Console.Write("Auto-tuning... ");
try
{
    byte[] t = await reader.AutoTuneLfSamplingAsync(5000);
    Console.WriteLine($"OK (0x{t[0]:X2})");
}
catch { Console.WriteLine("skipped"); }

Console.WriteLine("LF 태그를 올려 주세요. (Ctrl+C 종료)\n");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    // EM410X 시도
    try
    {
        byte[] uid = await reader.ReadLf125KhzUidAsync(600, cts.Token);
        Console.WriteLine($"[EM410X] {BitConverter.ToString(uid).Replace("-", "")}");
        await Task.Delay(300, cts.Token);
        continue;
    }
    catch (IksungProtocolException) { }
    catch (IksungTimeoutException) { }

    // ISO 11784 시도
    try
    {
        byte[] raw = await reader.ReadLfIso11784Async(600, cts.Token);
        ulong bits = 0;
        for (int i = 0; i < 8 && i < raw.Length; i++) bits |= ((ulong)raw[i]) << (i * 8);
        ulong animal = bits & 0x3FFFFFFFFF;
        ulong country = (bits >> 38) & 0x3FF;
        Console.WriteLine($"[ISO11784] Country={country:D3} Animal={animal:D12}");
        await Task.Delay(300, cts.Token);
        continue;
    }
    catch (IksungProtocolException) { }
    catch (IksungTimeoutException) { }

    try { await Task.Delay(50, cts.Token); } catch (OperationCanceledException) { break; }
}
```

09. AutoRead 이벤트 모드

개요

AutoRead 모드는 리더기가 자율적으로 카드를 감지하고, 감지 시 SDK를 통해 이벤트를 발생시킵니다. 애플리케이션이 폴링 루프를 직접 작성할 필요 없이 이벤트 핸들러만 등록하면 됩니다.

폴링 방식 vs AutoRead 비교

항목	폴링 방식	AutoRead 방식
CPU 사용	루프로 인해 높음	이벤트 기반, 낮음
응답 지연	폴링 간격에 따라 다름	즉각
구현 복잡도	while 루프 직접 작성	이벤트 핸들러만
동시 명령	루프 중 다른 명령 불가	이벤트 외 명령 가능

1. 기본 사용법

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

// 이벤트 핸들러 등록
reader.TagDetected += (sender, e) =>
{
    Console.WriteLine($"카드 감지!");
    Console.WriteLine($"  CardType : {e.CardType}");
    Console.WriteLine($"  UID      : {e.UidHex}");
};

// AutoRead 시작
await reader.StartAutoReadAsync();
Console.WriteLine("AutoRead 시작. 카드를 올려 주세요. (Enter로 중지)");

Console.ReadLine();

// AutoRead 중지
await reader.StopAutoReadAsync();
Console.WriteLine("AutoRead 중지.");
```


2. TagDetectedEventArgs 속성

속성	타입	설명
CardType	CardType	카드 종류 (ISO14443A, MifareClassic 등)
Uid	byte[]	UID 바이트 배열
UidHex	string	UID 16진수 문자열 (대시 없음)
Cmd1	byte	프로토콜 Major 명령 바이트
Cmd2	byte	프로토콜 Minor 명령 바이트
RawData	byte[]	원시 응답 데이터

3. CardType 열거형

```
public enum CardType
{
    Unknown,
    Iso14443a,      // ISO 14443 Type A (일반)
    Iso14443b,      // ISO 14443 Type B
    Iso14443ab,     // Type A 또는 B
    Iso15693,       // ISO 15693 HF 태그
    MifareClassic,  // Mifare Classic 1K/4K
    MifareUltralight, // Mifare Ultralight / NTag
    MifareDesfire,  // Mifare DESFire
    Felica,         // Sony FeliCa
    Lf125Khz,       // LF 125 kHz (EM410X 등)
}
```

4. 카드 타입별 처리

```

reader.TagDetected += (sender, e) =>
{
    string ts = DateTime.Now.ToString("HH:mm:ss.fff");

    switch (e.CardType)
    {
        case CardType.MifareClassic:
            Console.ForegroundColor = ConsoleColor.Cyan;
            Console.WriteLine($"{ts} [Mifare Classic] UID: {e.UidHex}");
            break;

        case CardType.MifareUltralight:
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine($"{ts} [Ultralight/NTag] UID: {e.UidHex}");
            break;

        case CardType.MifareDesfire:
            Console.ForegroundColor = ConsoleColor.Yellow;
            Console.WriteLine($"{ts} [DESFire] UID: {e.UidHex}");
            break;

        case CardType.Iso15693:
            Console.ForegroundColor = ConsoleColor.Magenta;
            Console.WriteLine($"{ts} [ISO 15693] UID: {e.UidHex}");
            break;

        case CardType.Lf125Khz:
            Console.ForegroundColor = ConsoleColor.DarkYellow;
            Console.WriteLine($"{ts} [LF 125kHz] UID: {e.UidHex}");
            break;

        default:
            Console.ForegroundColor = ConsoleColor.Gray;
            Console.WriteLine($"{ts} [{e.CardType}] UID: {e.UidHex}");
            break;
    }
    Console.ResetColor();
};

```

5. WPF에서 UI 스레드 마샬링

AutoRead 이벤트는 백그라운드 스레드에서 발생합니다.

WPF UI를 업데이트하려면 **Dispatcher** 를 통해 마샬링해야 합니다.

```
reader.TagDetected += (sender, e) =>
{
    // UI 스레드에서 실행
    Application.Current.Dispatcher.Invoke(() =>
    {
        UidTextBox.Text = e.UidHex;
        CardTypeLabel.Content = e.CardType.ToString();
    });
};
```

6. 이벤트 핸들러 등록 해제

AutoRead를 중지한 후에도 이벤트 구독이 남아 있으면 GC 누수가 발생할 수 있습니다.

```
EventHandler<TagDetectedEventArgs> handler = (sender, e) =>
{
    Console.WriteLine($"UID: {e.UidHex}");
};

reader.TagDetected += handler;
await reader.StartAutoReadAsync();

// ... 잠시 후 ...
await reader.StopAutoReadAsync();
reader.TagDetected -= handler; // 구독 해제
```

7. AutoRead 중 일반 명령 병행

AutoRead가 실행 중일 때도 일반 명령을 보낼 수 있습니다.

단, 동시 요청이 겹치지 않도록 주의하세요.

```
await reader.StartAutoReadAsync();

// AutoRead 중 버전 읽기 (가능)
string ver = await reader.ReadVersionAsync();
Console.WriteLine($"Version: {ver}");

await reader.StopAutoReadAsync();
```

10. ISO 7816 / USIM (SIM 카드)

개요

IS-3500K에는 USIM(SIM 카드) 슬롯이 내장되어 있습니다.

ISO 7816 T=0 프로토콜로 SIM 카드와 통신하여 ICCID, IMSI, 파일 시스템 등을 읽을 수 있습니다.

ISO 7816 기본 개념

용어	설명
ATR	Answer-to-Reset: 카드가 활성화 시 반환하는 초기 정보
TPDU	T=0 전송 단위, 5바이트 헤더 [CLA INS P1 P2 P3]
APDU	Application PDU: TPDU 위에 올라가는 애플리케이션 명령
SW	Status Word (2바이트): 9000=성공, 61XX=응답 대기 중
MF	Master File (3F00): 파일 시스템 루트
EF	Elementary File: 실제 데이터 파일

1. 카드 활성화 — ATR 수신

```
byte channel = 0; // 채널 번호 (0 고정, 슬롯 1개)
byte[] atr = await reader.UsimActivateAsync(channel, 3000);

Console.WriteLine($"ATR: {BitConverter.ToString(atr).Replace("-", " ")}");

// T=0 / T=1 프로토콜 확인
bool isT1 = atr.Length ≥ 2 && (atr[1] & 0x0F) == 1;
Console.WriteLine($"Protocol: {(isT1 ? "T=1" : "T=0")}");
```

2. APDU 전송 (TPDU)

```
// SELECT MF (Master File 3F00)
byte[] selectMf = [0x00, 0xA4, 0x00, 0x00, 0x02, 0x3F, 0x00];
byte[] resp = await reader.UsimSendTpduAsync(selectMf, channel, 2000);

Console.WriteLine($"SELECT MF: {BitConverter.ToString(resp).Replace("-", " ")}");
Console.WriteLine($"SW: {resp[^2]:X2}-{resp[^1]:X2}");
```

3. ICCID 읽기 예제

ICCID는 SIM 카드의 고유 식별 번호 (20자리) 입니다. EF_ICCID 파일 경로: **MF(3F00) > EF_ICCID(2FE2)**

```

byte[] atr = await reader.UsimActivateAsync(0, 3000);

// 1. SELECT EF_ICCID
byte[] selectIccid = [0x00, 0xA4, 0x00, 0x00, 0x02, 0x2F, 0xE2];
byte[] resp = await reader.UsimSendTpduAsync(selectIccid, 0, 2000);

bool canRead = resp.Length ≥ 2 && resp[^2] == 0x90 && resp[^1] == 0x00;

// GET RESPONSE가 필요한 경우 (SW = 9FXX)
if (!canRead && resp.Length ≥ 2 && resp[^2] == 0x9F)
{
    byte le = resp[^1];
    byte[] getResp = [0x00, 0xC0, 0x00, 0x00, le];
    resp = await reader.UsimSendTpduAsync(getResp, 0, 2000);
    canRead = resp.Length ≥ 2 && resp[^2] == 0x90;
}

if (canRead)
{
    // 2. READ BINARY (10바이트 = ICCID BCD)
    byte[] readBin = [0x00, 0xB0, 0x00, 0x00, 0x0A];
    resp = await reader.UsimSendTpduAsync(readBin, 0, 2000);

    if (resp.Length ≥ 12 && resp[^2] == 0x90)
    {
        byte[] bcd = resp[..10];
        // BCD nibble-swap 디코딩
        var sb = new System.Text.StringBuilder();
        foreach (byte b in bcd)
        {
            sb.Append((char)('0' + (b & 0x0F)));
            sb.Append((char)('0' + (b >> 4)));
        }
        Console.WriteLine($"ICCID: {sb}");
    }
}

await reader.UsimDeactivateAsync(0, 1000);

```

4. 카드 비활성화

```

await reader.UsimDeactivateAsync(channel, 1000);
Console.WriteLine("USIM deactivated.");

```

5. 시리얼 번호 읽기

```
byte[] serial = await reader.UsimReadSerialAsync();
Console.WriteLine($"USIM Serial: {BitConverter.ToString(serial).Replace("-", " ")");
```

6. 자주 사용하는 APDU 명령 참조

명령	CLA	INS	P1	P2	Lc/Data	설명
SELECT MF	00	A4	00	00	02 3F 00	루트 선택
SELECT EF_ICCID	00	A4	00	00	02 2F E2	ICCID 파일 선택
SELECT EF_IMSI	00	A4	00	00	02 6F 07	IMSI 파일 선택 (3GPP)
READ BINARY	00	B0	offset_hi	offset_lo	Le	파일 읽기
GET RESPONSE	00	C0	00	00	Le	응답 데이터 가져오기 (T=0)
VERIFY PIN	00	20	00	01	08 PIN...	PIN 검증

7. 전체 예제 — 반복 읽기

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] atr = await reader.UsimActivateAsync(0, 3000, cts.Token);
        Console.WriteLine($"ATR: {BitConverter.ToString(atr).Replace("-", " ") }");

        byte[] selectMf = [0x00, 0xA4, 0x00, 0x00, 0x02, 0x3F, 0x00];
        byte[] resp = await reader.UsimSendTpduAsync(selectMf, 0, 2000, cts.Token);
        Console.WriteLine($"SELECT MF SW: {resp[^2]:X2}{resp[^1]:X2}");

        await reader.UsimDeactivateAsync(0, 1000, cts.Token);
        Console.WriteLine("Done.\n");
    }
    catch (IksungTimeoutException) { Console.WriteLine("(no card)"); }
    catch (OperationCanceledException) { break; }

    try { await Task.Delay(5000, cts.Token); } catch (OperationCanceledException) { break; }
}
```


11. Bluetooth / BLE 설정

개요

IS-3500K에는 BLE(Bluetooth Low Energy) 모듈이 내장되어 있습니다.
이 API를 통해 BLE 장치 이름, TX 파워, GAP 연결 파라미터 등을 읽고 변경할 수 있습니다.

주의: BLE 이름 변경 등 설정 변경은 리더기 재시작 후에 적용됩니다.

1. 장치 이름 읽기 / 변경

```
// 현재 BLE 장치 이름 읽기
string name = await reader.BleReadNameAsync();
Console.WriteLine($"BLE Name: {name}");

// 장치 이름 변경
await reader.BleWriteNameAsync("MyReader-001");
Console.WriteLine("이름이 변경되었습니다. 재시작 후 적용됩니다.");

// 검증 읽기
string readback = await reader.BleReadNameAsync();
Console.WriteLine($"Readback: {readback}");
```

2. MAC 주소 읽기

```
byte[] mac = await reader.BleReadMacAddressAsync();
// MAC은 리틀 엔디언(LE) 순서로 반환되므로 역순으로 표시
string macStr = string.Join(":", mac.Reverse().Select(b => $"{b:X2}"));
Console.WriteLine($"MAC Address: {macStr}");
// 출력 예: "AB:CD:EF:12:34:56"
```

3. TX 파워 읽기 / 변경

TX 파워는 BLE 송신 강도입니다. 높을수록 도달 거리가 길어지지만 전력 소비가 증가합니다.

```
// TX 파워 읽기
byte[] txPwr = await reader.BleReadTxPowerAsync();
if (txPwr.Length > 0)
{
    string label = IksungReader.TxPowerToString(txPwr[0]);
    Console.WriteLine($"TX Power: {label} (index={txPwr[0]})");
}

// TX 파워 변경 (0~최대 인덱스, 제품마다 다름)
await reader.BleWriteTxPowerAsync(3); // 인덱스 3으로 설정
```

TxPowerToString 반환값 예시:

인덱스	레이블
0	"-40 dBm"
1	"-20 dBm"
2	"-16 dBm"
3	"-12 dBm"
4	"-8 dBm"
5	"-4 dBm"
6	"0 dBm"
7	"+4 dBm"

4. GAP 연결 파라미터 읽기

```
byte[] gap = await reader.BleReadGapConnectParamsAsync();
if (gap.Length ≥ 8)
{
    ushort minInterval = (ushort)((gap[0] << 8) | gap[1]);
    ushort maxInterval = (ushort)((gap[2] << 8) | gap[3]);
    ushort slaveLatency = (ushort)((gap[4] << 8) | gap[5]);
    ushort supTimeout = (ushort)((gap[6] << 8) | gap[7]);

    Console.WriteLine($"Min Interval : {minInterval} × 1.25ms = {minInterval * 1.25:F2}ms");
    Console.WriteLine($"Max Interval : {maxInterval} × 1.25ms = {maxInterval * 1.25:F2}ms");
    Console.WriteLine($"Slave Latency: {slaveLatency}");
    Console.WriteLine($"Sup Timeout : {supTimeout} × 10ms = {supTimeout * 10}ms");
}
```

5. Central 스캔 (주변 BLE 장치 탐색)

IS-3500K를 BLE Central로 동작시켜 주변 Peripheral 장치를 스캔합니다.

```
// Central 활성화 여부 확인
byte[] centralEn = await reader.BleReadCentralEnableAsync();
bool enabled = centralEn.Length > 0 && centralEn[0] == 1;
Console.WriteLine($"Central Mode: {(enabled ? "Enabled" : "Disabled")}");

// 스캔 시작
await reader.BleCentralScanStartAsync();
Console.WriteLine("스캔 중... (2초)");

await Task.Delay(2000);

// 스캔 중지
await reader.BleCentralScanStopAsync();

// 스캔 결과 목록 가져오기
try
{
    byte[] scanList = await reader.BleCentralScanListAsync(2000);
    if (scanList.Length == 0)
        Console.WriteLine("발견된 장치 없음.");
    else
        Console.WriteLine($"Scan raw: {BitConverter.ToString(scanList).Replace("-", " ")}");
}
catch (IksungProtocolException)
{
    Console.WriteLine("스캔 결과 없음 (미지원 또는 타임아웃).");
}
```

6. Peripheral 광고 (Advertising)

IS-3500K를 BLE Peripheral로 동작시켜 광고 패킷을 송출합니다.

```
// 광고 시작
await reader.BlePeripheralAdvertisingStartAsync();
Console.WriteLine("BLE 광고 시작.");

await Task.Delay(5000);

// 광고 중지
await reader.BlePeripheralAdvertisingStopAsync();
Console.WriteLine("BLE 광고 중지.");
```

7. 시스템 리셋

BLE 모듈을 소프트웨어적으로 재시작합니다.

주의: 이 명령 실행 시 리더기와의 연결이 끊어집니다.

```
Console.WriteLine("리셋 후 연결이 끊어집니다...");
await reader.BleSystemResetAsync();
// 이 이후 reader는 사용 불가
```

8. 전체 설정 조회 예제

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");
Console.WriteLine($"Firmware: {await reader.ReadVersionAsync()}\n");

// BLE 이름
Console.WriteLine($"BLE Name : {await reader.BleReadNameAsync()}");

// MAC 주소
byte[] mac = await reader.BleReadMacAddressAsync();
Console.WriteLine($"MAC Address: {string.Join(":", mac.Reverse().Select(b => $"{b:X2}")}");

// TX 파워
byte[] tx = await reader.BleReadTxPowerAsync();
if (tx.Length > 0)
    Console.WriteLine($"TX Power : {IksungReader.TxPowerToString(tx[0])}");

// GAP 파라미터
byte[] gap = await reader.BleReadGapConnectParamsAsync();
if (gap.Length ≥ 8)
{
    ushort min = (ushort)((gap[0] << 8) | gap[1]);
    ushort max = (ushort)((gap[2] << 8) | gap[3]);
    Console.WriteLine($"GAP Interval: {min * 1.25:F2}ms ~ {max * 1.25:F2}ms");
}

// Central 상태
byte[] ce = await reader.BleReadCentralEnableAsync();
Console.WriteLine($"Central Mode: {(ce.Length > 0 && ce[0] == 1 ? "Enabled" : "Disabled")}
```

12. 릴레이 보드 (Relay / Digital I/O)

개요

IS-3500K의 릴레이 보드는 디지털 입력과 릴레이 출력을 제어합니다.

채널	방향	수량	용도
DIN 1~5	입력	5채널	센서, 스위치, 검출기 연결
RELAY 1~8	출력	8채널	잠금장치, 전등, 경보음 제어

1. 디지털 입력 (DIN) 읽기

전체 채널 동시 읽기 (비트마스크)

```
byte[] result = await reader.RelayReadAllInputsAsync();
byte mask = result[0];

// bit0=DIN1, bit1=DIN2, ..., bit4=DIN5
for (byte i = 1; i ≤ 5; i++)
{
    bool state = IksungReader.GetInputState(mask, i);
    Console.WriteLine($"DIN {i}: {(state ? "HIGH" : "LOW")}");
}
```

개별 채널 읽기

```
byte[] result = await reader.RelayReadInputAsync(1); // DIN 1번
bool isHigh = result.Length > 0 && result[0] == 1;
Console.WriteLine($"DIN 1: {(isHigh ? "HIGH" : "LOW")}");
```

GetInputState 유틸리티

```
byte mask = (await reader.RelayReadAllInputsAsync())[0];

bool din1 = IksungReader.GetInputState(mask, 1);
bool din3 = IksungReader.GetInputState(mask, 3);
```

2. 릴레이 출력 (RELAY) 읽기

```
byte[] result = await reader.RelayReadAllOutputsAsync();
byte mask = result[0];

// bit0=RELAY1, ..., bit7=RELAY8
for (byte i = 1; i ≤ 8; i++)
{
    bool on = IksungReader.GetRelayState(mask, i);
    Console.WriteLine($"RELAY {i}: {(on ? "ON" : "OFF")}");
}
```

```
// 특정 릴레이 개별 읽기
byte[] result = await reader.RelayReadOutputAsync(3); // RELAY 3번
bool on = result.Length > 0 && result[0] == 1;
```

3. 릴레이 출력 제어

개별 릴레이 On/Off

```
await reader.RelayWriteOutputAsync(1, true); // RELAY 1 → ON
await reader.RelayWriteOutputAsync(1, false); // RELAY 1 → OFF
```

전체 릴레이 비트마스크 쓰기

```
// RELAY 1, 3, 5 ON / 나머지 OFF
byte mask = 0b_0001_0101; // bit0=1, bit2=1, bit4=1
await reader.RelayWriteAllOutputsAsync(mask);
```

전체 ON / 전체 OFF

```
await reader.RelayAllOnAsync(); // 모든 릴레이 ON
await Task.Delay(1000);
await reader.RelayAllOffAsync(); // 모든 릴레이 OFF
```

4. Auto-Off 타이머

릴레이를 켜 후 지정한 시간이 지나면 자동으로 꺼지도록 설정합니다.

```
byte relayNo = 1;
ushort delayMs = 2000; // 2초 후 자동 OFF

await reader.RelaySetAutoOffTimeAsync(relayNo, delayMs);
await reader.RelayWriteOutputAsync(relayNo, true);
Console.WriteLine($"RELAY {relayNo} ON → {delayMs}ms 후 자동 OFF");

await Task.Delay(delayMs + 500);

byte[] result = await reader.RelayReadAllOutputsAsync();
bool stillOn = IksungReader.GetRelayState(result[0], relayNo);
Console.WriteLine($"RELAY {relayNo}: {(stillOn ? "아직 ON" : "자동 OFF 완료")}");
```

5. 입력 실시간 모니터링

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

Console.WriteLine("DIN 입력 모니터링 (Ctrl+C 종료)...\n");

byte prevMask = 0xFF;

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] res = await reader.RelayReadAllInputsAsync(500, cts.Token);
        byte mask = res.Length > 0 ? res[0] : (byte)0;

        if (mask != prevMask)
        {
            string ts = DateTime.Now.ToString("HH:mm:ss.fff");
            Console.Write($"{ts} ");
            for (byte i = 1; i <= 5; i++)
                Console.Write($"DIN{i}:{(IksungReader.GetInputState(mask, i) ? "H" : "L")}");
            Console.WriteLine($" (0x{mask:X2})");
            prevMask = mask;
        }

        await Task.Delay(50, cts.Token);
    }
    catch (OperationCanceledException) { break; }
    catch (IksungTimeoutException) { await Task.Delay(200, cts.Token); }
}
```

6. 순차 릴레이 제어 예제


```

await using var reader = await IksungReader.ConnectSerialAsync("COM3");

// 1→8 순차 ON
for (byte i = 1; i ≤ 8; i++)
{
    await reader.RelayWriteOutputAsync(i, true);
    Console.WriteLine($"RELAY {i} ON");
    await Task.Delay(200);
}

// 전체 OFF
await reader.RelayAllOffAsync();
Console.WriteLine("All OFF");
await Task.Delay(500);

// 8→1 역순 ON
for (byte i = 8; i ≥ 1; i--)
{
    await reader.RelayWriteOutputAsync(i, true);
    Console.WriteLine($"RELAY {i} ON");
    await Task.Delay(200);
}

// 전체 OFF
await reader.RelayAllOffAsync();
Console.WriteLine("All OFF");

```

7. API 요약

메서드	설명
<code>RelayReadAllInputsAsync()</code>	DIN 1~5 전체 비트마스크 읽기
<code>RelayReadInputAsync(inputNo)</code>	특정 DIN 채널 읽기
<code>RelayReadAllOutputsAsync()</code>	RELAY 1~8 전체 비트마스크 읽기
<code>RelayReadOutputAsync(relayNo)</code>	특정 RELAY 채널 읽기
<code>RelayWriteAllOutputsAsync(mask)</code>	전체 RELAY 비트마스크로 설정
<code>RelayWriteOutputAsync(no, on)</code>	개별 RELAY On/Off
<code>RelayAllOnAsync()</code>	전체 RELAY ON
<code>RelayAllOffAsync()</code>	전체 RELAY OFF

메서드	설명
<code>RelaySetAutoOffTimeAsync(no, ms)</code>	Auto-Off 타이머 설정
<code>GetInputState(mask, inputNo)</code>	비트마스크에서 DIN 상태 추출 (static)
<code>GetRelayState(mask, relayNo)</code>	비트마스크에서 RELAY 상태 추출 (static)

13. 예외 처리 및 고급 사용

1. 예외 종류

SDK는 두 가지 전용 예외를 사용합니다.

IksungProtocolException

리더기가 오류 응답(State ≠ 0)을 반환했을 때 발생합니다.

원인 예시: - 카드가 응답하지 않음 (RF 범위 이탈) - 인증 실패 (잘못된 키) - 지원하지 않는 명령 - 카드 Lock 상태에서 쓰기 시도

```
using Iksung.Reader.Exceptions;

try
{
    await reader.MifareAuthenticateAsync(4, MifareKeyType.KeyA, wrongKey);
}
catch (IksungProtocolException ex)
{
    Console.WriteLine($"프로토콜 오류: {ex.Message}");
    // ex.State: 리더기에서 반환된 State 바이트
    Console.WriteLine($"State code: 0x{ex.State:X2}");
}
```

IksungTimeoutException

지정한 `timeoutMs` 내에 리더기가 응답하지 않을 때 발생합니다.

원인 예시: - 카드가 없음 - 리더기가 응답하지 않음 - `timeoutMs` 값이 너무 짧음

```
try
{
    byte[] uid = await reader.ReadIso14443aUidAsync(500);
}
catch (IksungTimeoutException ex)
{
    // 카드 없음 - 정상적인 상황이므로 보통 무시
    // Console.WriteLine($"타임아웃: {ex.Message}");
}
```

2. 권장 예외 처리 패턴

폴링 루프

```
while (!cts.Token.IsCancellationRequested)
{
    try
    {
        byte[] uid = await reader.ReadIso14443aUidAsync(500, cts.Token);
        // 카드 처리 로직
        await Task.Delay(300, cts.Token);
    }
    catch (IksungTimeoutException) { /* 카드 없음 - 계속 */ }
    catch (IksungProtocolException ex) { Console.WriteLine($"오류: {ex.Message}"); }
    catch (OperationCanceledException) { break; }
}
```

일회성 작업

```
try
{
    byte[] atr = await reader.ActivateMifareAsync();
    await reader.MifareAuthenticateAsync(4, MifareKeyType.KeyA, key);
    byte[] data = await reader.MifareReadBlockAsync(4);
    Console.WriteLine(BitConverter.ToString(data));
}
catch (IksungProtocolException ex)
{
    Console.WriteLine($"카드 오류: {ex.Message}");
}
catch (IksungTimeoutException)
{
    Console.WriteLine("카드를 인식할 수 없습니다.");
}
```

3. 취소 토큰 (CancellationToken) 사용

모든 async 메서드는 `CancellationToken` 을 지원합니다.

```

using var cts = new CancellationTokenSource();

// Ctrl+C 처리
Console.CancelKeyPress += (_, e) =>
{
    e.Cancel = true;
    cts.Cancel();
};

// 3초 후 자동 취소
using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(3));

try
{
    byte[] uid = await reader.ReadIso14443aUidAsync(2000, timeoutCts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("사용자가 취소하거나 타임아웃되었습니다.");
}

```

4. Raw 명령 전송

표준 API에 없는 명령을 직접 보낼 때 사용합니다.

프로토콜 탐색, 신규 펌웨어 기능 테스트에 유용합니다.

```

byte cmd1 = 0x00; // Major: MAJOR_COMMON
byte cmd2 = 0x10; // Minor: READ_VERSION

byte[]? data = null; // 데이터 없음

byte[] response = await reader.SendRawCommandAsync(cmd1, cmd2, data, 1000);
Console.WriteLine($"Response: {BitConverter.ToString(response).Replace("-", " ")}");

```

```

// 데이터 포함 예시
byte[] payload = [0x01, 0x02, 0x03];
byte[] resp = await reader.SendRawCommandAsync(0x02, 0x30, payload, 2000);

```

주요 프로토콜 상수 (Cmd1 참조)

CMD1 (Hex)	대상
0x00	공통 (버전, UID, RF)

CMD1 (Hex)	대상
0x01	ISO 14443 A/B
0x02	Mifare Classic
0x03	ISO 15693
0x05	Mifare Ultralight / NTag
0x09	DESFire
0x0A	ISO 7816 / USIM
0x0C	LF 125 kHz
0x12	BLE 설정
0x20	AutoRead
0x22	릴레이 I/O
0x23	릴레이 설정

5. 연결 상태 확인

```
Console.WriteLine($"Connected : {reader.IsConnected}");
Console.WriteLine($"Via      : {reader.ConnectedVia}"); // Serial, Socket 등
```

6. 비동기 디스패치 — WPF / MAUI

이벤트(**TagDetected**)와 async 완료 콜백은 백그라운드 스레드에서 호출됩니다.
UI 프레임워크의 컨트롤은 UI 스레드에서만 수정 가능합니다.

WPF:

```
reader.TagDetected += (s, e) =>
{
    Application.Current?.Dispatcher.Invoke(() =>
    {
        MyTextBlock.Text = e.UidHex;
    });
};
```

MAUI / Avalonia (MainThread):

```
reader.TagDetected += (s, e) =>
{
    MainThread.BeginInvokeOnMainThread(() =>
    {
        UidLabel.Text = e.UidHex;
    });
};
```

7. 성능 팁

상황	권장사항
카드 감지만 필요	<code>AutoRead</code> 모드 사용 (폴링 루프보다 CPU 절약)
빠른 연속 읽기	<code>timeoutMs</code> 를 짧게 (300~500ms), 딜레이 최소화
블록 여러 개 읽기	<code>Iso15693ReadMultipleBlocksAsync</code> 또는 <code>NTagFastReadAsync</code> 활용
다중 스레드	<code>IksungReader</code> 하나는 단일 스레드에서 직렬 사용 권장
정기 구독	<code>TagDetected</code> 이벤트를 사용 후 <code>--</code> 로 해제

14. 시스템 레벨 기능

연결 생명주기 관리, 상태 이벤트, 진단, 포트 탐색, Raw 패킷 모니터링 등 SDK의 시스템 레벨 기능을 설명합니다.

목차

기능	클래스/멤버
A. 포트 탐색	<code>IksungReaderDiscovery</code>
B. 연결 옵션	<code>IksungReaderOptions</code>
C. 연결 상태 이벤트	<code>reader.ConnectionChanged</code>
D. 리더기 정보 조회	<code>reader.GetReaderInfoAsync</code> , <code>reader.PingAsync</code>
E. Raw 패킷 모니터링	<code>reader.RawPacketReceived</code>
F. 자동 재연결	<code>IksungReaderOptions.AutoReconnect</code>
G. 명시적 연결 해제	<code>reader.DisconnectAsync</code>

A. 포트 탐색

`IksungReaderDiscovery.GetAllSerialPorts()`

시스템에 등록된 모든 시리얼 포트 이름을 반환합니다. `System.IO.Ports.SerialPort.GetPortNames()` 래핑입니다.

```
IEnumerable<string> ports = IksungReaderDiscovery.GetAllSerialPorts();
foreach (var p in ports)
    Console.WriteLine(p); // COM3, COM4, ...
```

`IksungReaderDiscovery.ScanIksungPortsAsync()`

각 포트에 Version 패킷을 전송하여 **Iksung 프로토콜로 응답하는 포트만** 반환합니다. 최대 4개 포트를 병렬로 테스트하므로 빠릅니다.


```
var progress = new Progress<(string portName, bool isIksung)>(p =>
    Console.WriteLine($"{p.portName}: {(p.isIksung ? "✓ Iksung" : "X")}"));

IReadOnlyList<string> found = await IksungReaderDiscovery.ScanIksungPortsAsync(
    baudRate:      115200,    // 기본값
    pingTimeoutMs:  400,      // 포트당 대기 시간
    progressCallback: progress);

Console.WriteLine($"발견된 포트: {string.Join(", ", found)}");
```

IksungReaderDiscovery.PingPortAsync()

단일 포트에 Version 패킷을 보내 Iksung 리더기인지 확인합니다.

이미 다른 프로세스가 점유 중인 포트는 예외 없이 `false` 를 반환합니다.

```
bool isIksung = await IksungReaderDiscovery.PingPortAsync("COM3", baudRate: 115200);
Console.WriteLine(isIksung ? "Iksung 리더기" : "응답 없음");
```

B. 연결 옵션

`IksungReaderOptions` 클래스로 연결 동작을 제어합니다.

```
var options = new IksungReaderOptions
{
    AutoReconnect      = true,    // 비정상 끊김 시 자동 재연결
    ReconnectDelayMs   = 2000,    // 재시도 간격 (ms)
    DefaultTimeoutMs   = 1000,    // PingAsync 등 기본 타임아웃
    LogRawPackets       = false,   // RawPacketReceived 이벤트 발생 여부
};
```

팩토리 메서드에 전달

```
// 연결과 동시에 옵션 적용
await using var reader = await IksungReader.ConnectSerialAsync("COM3", 115200, options);
```

연결 후 적용

```
// 기존 연결 유지하면서 옵션 변경
await using var reader = await IksungReader.ConnectSerialAsync("COM3");
reader.ApplyOptions(new IksungReaderOptions { AutoReconnect = true });
```

속성	타입	기본값	설명
<code>AutoReconnect</code>	<code>bool</code>	<code>false</code>	비정상 끊김 시 자동 재연결
<code>ReconnectDelayMs</code>	<code>int</code>	<code>2000</code>	재연결 시도 간격 (ms)
<code>DefaultTimeoutMs</code>	<code>int</code>	<code>1000</code>	<code>PingAsync</code> 등 기본 타임아웃
<code>LogRawPackets</code>	<code>bool</code>	<code>false</code>	<code>RawPacketReceived</code> 이벤트 활성화

C. 연결 상태 이벤트

`reader.ConnectionChanged` 이벤트는 연결/끊김 시 발생합니다.

```
reader.ConnectionChanged += (sender, isConnected) =>
{
    if (isConnected)
        Console.WriteLine("✓ 연결됨");
    else
        Console.WriteLine("X 연결 끊김 - 재연결 시도 중...");
};
```

주의: 이벤트 핸들러는 **백그라운드 스레드**에서 호출됩니다.

WinForms/WPF에서 UI 컨트롤에 접근하려면 `Invoke` / `Dispatcher.Invoke` 가 필요합니다.

WinForms 예제

```
// .NET Framework 4.x 이상
reader.ConnectionChanged += (sender, isConnected) =>
{
    if (InvokeRequired)
        Invoke(new Action(() => UpdateStatus(isConnected)));
    else
        UpdateStatus(isConnected);
};
```

WPF 예제

```
reader.ConnectionChanged += (sender, isConnected) =>
    Application.Current.Dispatcher.Invoke(() => UpdateStatus(isConnected));
```

D. 리더기 정보 조회

`reader.IsConnected`

현재 연결 여부를 즉시 반환하는 속성입니다.

```
if (reader.IsConnected)
    Console.WriteLine("연결됨");
```

`reader.PingAsync()`

리더기에 Version 요청을 보내 응답 여부를 확인합니다.

연결 안 됨, 타임아웃, 예외 — 모두 예외 없이 `false` 를 반환합니다.

```
bool alive = await reader.PingAsync(timeoutMs: 500);
Console.WriteLine(alive ? "OK" : "응답 없음");

// DefaultTimeoutMs 사용 (timeoutMs 생략)
bool alive2 = await reader.PingAsync();
```

`reader.GetReaderInfoAsync()`

Version 요청 결과를 `ReaderInfo` 구조체로 반환합니다.

```
ReaderInfo info = await reader.GetReaderInfoAsync();
Console.WriteLine(info);
// 출력 예: [Serial] COM3 @ 115200 bps | FW: IS-3500K_V1.8 | Connected
```

속성	타입	설명
<code>FirmwareVersion</code>	<code>string</code>	펌웨어 버전 문자열
<code>Channel</code>	<code>ChannelType</code>	연결 채널 (Serial / Socket)
<code>IsConnected</code>	<code>bool</code>	현재 연결 여부
<code>PortOrAddress</code>	<code>string?</code>	포트 이름 또는 IP:Port

E. Raw 패킷 모니터링

`IksungReaderOptions.LogRawPackets = true` 로 활성화하면 모든 송신(TX)·수신(RX) 패킷의 raw 바이트를 이벤트로 받습니다.

```
var options = new IksungReaderOptions { LogRawPackets = true };
await using var reader = await IksungReader.ConnectSerialAsync("COM3", 115200, options);

reader.RawPacketReceived += (sender, e) =>
{
    string dir = e.IsTransmit ? "TX" : "RX";
    string hex = BitConverter.ToString(e.Data).Replace("-", " ");
    Console.WriteLine($"[{e.Timestamp:HH:mm:ss.fff}] {dir}: {hex}");
};

// ReadVersion 실행 시 TX/RX 패킷이 출력됩니다
string version = await reader.ReadVersionAsync();
```

출력 예:

```
[14:30:01.123] TX: 01 00 10 00 00 10 03
[14:30:01.131] RX: 01 00 10 01 00 0F 49 53 2D 33 35 30 30 4B 5F 56 31 2E 38 FF 03
```

속성	타입	설명
<code>Data</code>	<code>byte[]</code>	on-wire raw 바이트
<code>IsTransmit</code>	<code>bool</code>	<code>true</code> =TX, <code>false</code> =RX
<code>Timestamp</code>	<code>DateTime</code>	캡처 시각

성능 주의: 패킷마다 `byte[]` 복사본이 생성됩니다.
고속 AutoRead 환경에서는 불필요할 때 비활성화하세요.

F. 자동 재연결

`IksungReaderOptions.AutoReconnect = true` 이면,
케이블 뽑힘 등 **비정상 끊김** 감지 시 자동으로 재연결을 시도합니다.

- `ReconnectDelayMs` 마다 재시도
- 재연결 성공 시 `ConnectionChanged(true)` 이벤트 발생
- `DisconnectAsync()` 를 명시적으로 호출하면 자동 재연결 **중단**

```
var options = new IksungReaderOptions
{
    AutoReconnect    = true,
    ReconnectDelayMs = 2000,    // 2초마다 재시도
};

await using var reader = await IksungReader.ConnectSerialAsync("COM3", 115200, options);

reader.ConnectionChanged += (_, connected) =>
    Console.WriteLine(connected ? "✓ 재연결됨!" : "X 연결 끊김 - 재시도 중");
```

AutoReconnect 적용 채널 — V1 현재 Serial 채널에서 검증되어 있습니다. PC/SC 채널은 OS의 카드/리더 상태 변화에 따라 동작이 달라질 수 있으므로 끊김 감지 후 `ConnectPcscAsync`를 직접 호출하는 패턴도 함께 고려하세요.

G. 명시적 연결 해제

`reader.DisconnectAsync()`는 `AutoReconnect`가 켜져 있어도 재연결을 방지합니다.

```
// 케이블 교체, 포트 변경 전 명시적 해제
await reader.DisconnectAsync();
Console.WriteLine("연결 해제됨 (AutoReconnect 중단)");

// await using 패턴에서는 DisposeAsync가 자동으로 해제
await using var reader = await IksungReader.ConnectSerialAsync("COM3");
// ... 블록 종료 시 자동 해제 + AutoReconnect 중단
```

전체 예제

```
// 포트 자동 탐색 + 연결 + 모니터링
var ports = await IksungReaderDiscovery.ScanIksungPortsAsync();
if (ports.Count == 0) { Console.WriteLine("리더기 없음"); return; }

var options = new IksungReaderOptions
{
    AutoReconnect = true,
    LogRawPackets = true,
};

await using var reader = await IksungReader.ConnectSerialAsync(ports[0], 115200, options)

reader.ConnectionChanged += (_, ok) =>
    Console.WriteLine(ok ? "연결됨" : "끊김 - 재연결 중");

reader.RawPacketReceived += (_, e) =>
    Console.WriteLine($"{(e.IsTransmit ? "TX" : "RX")}: {BitConverter.ToString(e.Data)}");

ReaderInfo info = await reader.GetReaderInfoAsync();
Console.WriteLine(info);

bool ok2 = await reader.PingAsync();
Console.WriteLine($"Ping: {ok2}");
```

관련 샘플

샘플	경로
NET8	NET8-Samples/14-SystemFeatures/
NET4x	NET4x-Samples/14-SystemFeatures/

```
# 빌드 및 실행 (NET8)
dotnet run --project NET8-Samples/14-SystemFeatures -- COM3
dotnet run --project NET8-Samples/14-SystemFeatures -- scan

# 빌드 및 실행 (NET4x)
dotnet build NET4x-Samples/14-SystemFeatures/SystemFeatures.Net4x.csproj
NET4x-Samples/14-SystemFeatures/bin/Debug/net472/SystemFeatures.Net4x.exe COM3
NET4x-Samples/14-SystemFeatures/bin/Debug/net472/SystemFeatures.Net4x.exe scan
```

← 목차

PC/SC (CCID) 채널 사용 가이드

1. 개요

PC/SC(Personal Computer/Smart Card) 채널은 USB CCID 모드로 동작하는 IS-NFC 시리즈 리더기를 Windows Smart Card Service(SCardSvr)를 통해 제어하는 방법입니다.

IS-3500Z, IS-3500K 등 USB CCID 인터페이스를 갖춘 리더기는 시리얼 케이블 없이 USB 케이블만으로 PC에 연결할 수 있으며, 별도 드라이버 설치 없이 Windows 내장 USB HID/CCID 드라이버로 동작합니다.

Iksung.Reader SDK는 내부적으로 `winscard.dll` P/Invoke를 사용하여 PC/SC 통신을 처리하므로 외부 NuGet 패키지 의존성이 없습니다. STX/ETX 패킷과 ISO 7816-4 APDU 사이의 변환도 SDK 내부에서 자동으로 수행되므로 상위 레이어 코드는 Serial/Socket 채널과 동일하게 작성할 수 있습니다.

2. 요구사항

항목	요구 사항
운영체제	Windows 10 / Windows 11 (Windows 전용)
.NET 버전	.NET 8 이상, 또는 .NET Framework 4.7.2 이상
TargetFramework	<code>net8.0-windows</code> 또는 <code>net472</code>
Smart Card Service	<code>SCardSvr</code> (스마트 카드 서비스) 실행 중
빌드 플랫폼	x86 권장 (winscard.dll 호환성)

Smart Card Service 활성화 확인 방법

Win + R → `services.msc` → "스마트 카드" 검색 → 시작 유형: 자동, 상태: 실행 중 확인

또는 PowerShell:

```
Get-Service SCardSvr | Select-Object Status, StartType
# Status: Running, StartType: Manual 또는 Automatic 이어야 합니다.

# 서비스 시작 (관리자 권한 필요)
Start-Service SCardSvr
```


3. 리더 검색

`IksungPcscDiscovery.GetAvailableReaders()` 를 사용하여 현재 시스템에 연결된 PC/SC 리더 목록을 조회합니다.

```
using Iksung.Reader;

var readers = IksungPcscDiscovery.GetAvailableReaders();

if (readers.Count == 0)
{
    Console.WriteLine("연결된 리더가 없습니다.");
    return;
}

foreach (var name in readers)
    Console.WriteLine(name);
// 출력 예: "iksung IS-3500Z 0"
//          "ACS ACR39U ICC Reader 0"
```

Smart Card Service 가 중지된 경우 빈 목록을 반환합니다 (예외를 던지지 않음).

4. 리더 Hot-plug 모니터링

`IksungPcscDiscovery.StartMonitor()` 를 호출하면 백그라운드에서 1초마다 리더 목록을 폴링하고, 변경이 감지되면 `ReaderListChanged` 이벤트를 발사합니다.

```
// 모니터링 시작 (애플리케이션 시작 시)
IksungPcscDiscovery.ReaderListChanged += (_, readers) =>
{
    Console.WriteLine($"리더 목록 변경: {readers.Count}개");
    foreach (var r in readers)
        Console.WriteLine($" {r}");
};
IksungPcscDiscovery.StartMonitor(pollIntervalMs: 1000);

// 모니터링 중지 (애플리케이션 종료 시)
IksungPcscDiscovery.StopMonitor();
```

폴링 간격(`pollIntervalMs`)의 기본값은 1000ms(1초)입니다. WPF/WinForms 앱에서 이벤트 핸들러 안에서 UI를 업데이트할 경우, 이벤트는 백그라운드 스레드에서 발사되므로 `Dispatcher.Invoke` 또는 `BeginInvoke` 가 필요합니다.

5. 연결

`IksungReader.ConnectPcscAsync()` 팩토리 메서드로 PC/SC 리더에 연결합니다.

```
using Iksung.Reader;
using Iksung.Reader.Exceptions;

string readerName = "iksung IS-3500Z 0"; // GetAvailableReaders() 결과에서 선택

try
{
    await using var reader = await IksungReader.ConnectPcscAsync(readerName);
    Console.WriteLine($"연결 성공! 채널: {reader.ConnectedVia}"); // ChannelType.Pcsc

    // 이후 명령 전송
    var info = await reader.GetReaderInfoAsync();
    Console.WriteLine($"FW: {info.FirmwareVersion}");
}
catch (ChannelDisconnectedException ex)
{
    Console.WriteLine($"연결 실패: {ex.Message}");
}
```

연결 옵션(`IksungReaderOptions`)을 지정할 수도 있습니다:

```
var options = new IksungReaderOptions { LogRawPackets = true };
await using var reader = await IksungReader.ConnectPcscAsync(readerName, options);
```

주의: 일부 PC/SC 리더는 카드/태그가 삽입된 상태에서만 `SCardConnect` 가 성공합니다. 리더 모델에 따라 다르므로 카드를 올려놓은 상태에서 연결을 시도하세요.

6. ATR 조회

ATR(Answer To Reset)은 카드가 리셋될 때 반환하는 정보 바이트로, 카드 타입과 프로토콜을 식별합니다. SDK는 `SCardConnect` 직후 `SCardStatus` 를 통해 ATR을 자동으로 캐싱합니다.

`ConnectedVia == ChannelType.Pcsc` 임을 확인한 후 내부 채널에 접근하는 방법은 현재 V1 공개 API에서는 제공되지 않습니다. ATR이 필요한 경우 `SendRawCommandAsync` 를 사용하거나 SDK 향후 버전에서 제공 예정인 `reader.GetAtrAsync()` 를 사용하세요.

연결 성공 자체로 카드 타입이 인식된 것으로 간주하고, 이후 명령으로 카드 정보를 조회하는 것을 권장합니다.

7. 기본 명령 실행

PC/SC 채널로 연결한 후에는 Serial/Socket 채널과 동일한 API를 사용합니다. SDK 내부에서 STX/ETX 패킷을 ISO 7816-4 APDU로 자동 변환합니다.

```
await using var reader = await IksungReader.ConnectPcscAsync(readerName);

// 리더 정보 읽기 (FirmwareVersion · Channel · IsConnected · PortOrAddress)
ReaderInfo info = await reader.GetReaderInfoAsync();
Console.WriteLine($"FW: {info.FirmwareVersion} | 채널: {info.Channel} | 리더: {info.PortOrAddress}");

// 카드 UID 읽기 (카드가 리더 위에 있어야 함 · 카드 종류 자동 감지)
// ReadAllUidAsync 는 byte[] 1건을 반환합니다 (현재 필드에 있는 카드의 UID)
byte[] uid = await reader.ReadAllUidAsync();
string hex = BitConverter.ToString(uid).Replace("-", " ");
Console.WriteLine($"UID: {hex} ({uid.Length} bytes)");

// 카드 종류를 별도로 조회하고 싶을 때
byte[] cardType = await reader.ReadAllCardTypeAsync();
Console.WriteLine($"카드 타입 코드: {BitConverter.ToString(cardType)}");

// 부저
await reader.BuzzerAsync(); // 기본 200ms (durationUnit=2)
await reader.BuzzerAsync(5); // 500ms

// RF 제어
await reader.RfOffAsync();
await Task.Delay(500);
await reader.RfOnAsync();
```

8. ISO 7816-4 APDU 변환 규칙

SDK 내부에서 STX/ETX 패킷을 ISO 7816-4 APDU로 자동 변환합니다. 상위 레이어에서는 이 변환을 신경 쓸 필요가 없으나, 로우 레벨 디버깅 시 참고하세요.

Case	조건	APDU 구조	설명
Case 2 (Short)	요청 Data 없음	FF CMD1 CMD2 00 00	Le=00: 최대 256byte 응답 기대
Case 3 (Short)	요청 Data 있음, ≤255 bytes	FF CMD1 CMD2 00 Lc Data	SW만 응답

Case	조건	APDU 구조	설명
Case 3 (Extended)	요청 Data 있음, >255 bytes	FF CMD1 CMD2 00 00 LcH LcL Data	부트로더 전용

- **CLA = 0xFF** — IKSUNG 벤더 클래스
- **INS = CMD1** — 명령 Major 바이트
- **P1 = CMD2** — 명령 Minor 바이트 (buzzer 비트 포함)
- **P2 = 0x00**

응답 마지막 2바이트는 SW1/SW2입니다. **SW = 90 00** 이 아니면 **IksungProtocolException** 을 던집니다.

9. AutoRead

PC/SC 는 request-response 모델로만 동작합니다. 리더기가 자발적으로 패킷을 푸시하는 Serial AutoRead 모드와 달리, PC/SC 에서의 AutoRead 는 SDK 내부 폴링으로 구현됩니다.

```
reader.TagDetected += (_, e) =>
{
    // TagDetectedEventArgs:
    // - CardType : 감지된 카드/태그 종류 (CardType enum)
    // - Uid       : byte[] UID
    // - UidHex    : string (하이픈 없이 대문자, 편의용)
    // - Cmd1/Cmd2 : 응답 cmd 바이트 (raw)
    // - RawData   : 펌웨어 응답 전체 페이로드
    Console.WriteLine($"태그 감지! UID={e.UidHex} (타입: {e.CardType})");
};

await reader.StartAutoReadAsync();
// ... 대기 (예: Console.ReadKey)
await reader.StopAutoReadAsync();
```

V1 제약사항: PC/SC 채널에서 AutoRead 는 **ReadAllUid** 명령을 반복 폴링하는 방식으로 동작합니다. Serial 채널의 하드웨어 push 방식보다 응답 지연이 있을 수 있습니다. V2에서 **SCardGetStatusChange** 이벤트 모델 기반 개선이 계획되어 있습니다.

10. 펌웨어 업데이트 / Raw 명령

PC/SC 채널도 Serial 과 동일하게 `SendRawCommandAsync` 를 통해 임의의 STX/ETX 명령(CMD1·CMD2·Data) 을 직접 송신할 수 있습니다. SDK 내부에서 ISO 7816-4 APDU 로 자동 변환됩니다 (§ 8 참조).

```
await using var reader = await IksungReader.ConnectPcscAsync(readerName);

// 예: Common 버전 명령을 raw 로 직접 송신
byte[] resp = await reader.SendRawCommandAsync(
    cmd1: 0x00,      // MAJOR_COMMON
    cmd2: 0x00,      // COMMON_VERSION
    data: null,
    timeoutMs: 1000);
Console.WriteLine($"버전: {System.Text.Encoding.ASCII.GetString(resp).Trim()}");
```

펌웨어 업데이트 — IS-3500K V6.0/V7.0 부트로더 기반 양산 펌웨어 업데이트는 별도 도구 (`IksungFirmwareUpdater`) 가 내부적으로 처리하며, 일반 응용에서 Raw I/O 모드를 직접 다룰 필요는 없습니다.

11. 오류 처리

예외	원인	대처 방법
<code>ChannelDisconnectedException</code>	PC/SC 연결 실패 / 끊김	리더 이름 확인, 카드 삽입 여부 확인
<code>IksungProtocolException</code>	SW ≠ 90 00	리더기 로그 확인, 명령 파라미터 검토
<code>IksungTimeoutException</code>	응답 타임아웃	타임아웃 값 증가, 카드 위치 확인
<code>OperationCanceledException</code>	CancellationToken 취소	정상 취소 흐름

Smart Card Service 관련 오류 코드:

HRESULT	의미
<code>0x8010001D SCARD_E_NO_SERVICE</code>	Smart Card Service 중지됨
<code>0x80100007 SCARD_E_SERVICE_STOPPED</code>	Service 중지됨 (연결 중 발생)
<code>0x8010002E SCARD_E_NO_READERS_AVAILABLE</code>	리더 없음

이 경우 `GetAvailableReaders()` 는 예외 없이 빈 목록을 반환합니다.

12. 문제 해결 (FAQ)

Q. `GetAvailableReaders()` 가 빈 목록을 반환합니다.

- USB CCID 리더가 연결되어 있는지 확인하세요.
- 장치 관리자에서 "스마트 카드 판독기" 범주에 리더가 표시되는지 확인하세요.
- Smart Card Service(`SCardSvr`)가 실행 중인지 확인하세요.
- Windows가 CCID 드라이버를 인식하지 못하는 경우 USB 포트를 변경해 보세요.

Q. `ConnectPcscAsync` 가 실패합니다 (`ChannelDisconnectedException`).

- PC/SC 리더는 카드가 삽입된 상태에서만 `SCardConnect` 에 성공하는 경우가 있습니다. IS-3500Z 등 NFC 리더는 태그를 리더 위에 올려놓은 상태에서 연결을 시도하세요.

Q. AutoRead 이벤트가 느립니다.

- PC/SC 채널은 폴링 방식이므로 Serial 채널보다 지연이 있습니다. 태그 감지 간격을 줄이려면 AutoRead 폴링 주기 설정을 확인하세요.

Q. `net8.0` 으로 빌드하면 컴파일 오류가 발생합니다.

- `winscard.dll` P/Invoke 코드는 Windows 전용입니다. 프로젝트의 `TargetFramework` 를 `net8.0-windows` 로 변경하세요.

Q. 64비트 빌드에서 동작하지 않습니다.

- `PlatformTarget` 을 `x86` 으로 설정하면 32비트로 빌드되어 호환성이 높습니다. 64비트로 빌드하는 경우 `winscard.dll` 경로(`System32` vs `SysWOW64`)를 확인하세요. 일반적으로 x86 빌드를 권장합니다.

Q. 여러 리더가 있을 때 특정 리더에 연결하려면?

```
var readers = IksungPcscDiscovery.GetAvailableReaders();
// 이름에 "iksung" 이 포함된 첫 번째 리더 선택
string? target = readers.FirstOrDefault(r => r.Contains("iksung"));
if (target != null)
    await using var reader = await IksungReader.ConnectPcscAsync(target);
```

API 레퍼런스 (전체 메서드 목록)

모든 공개(public) 메서드의 서명과 간략한 설명입니다.

메서드 이름 뒤 **Async** 는 생략하지 않았습니다.

IksungReader — 팩토리 / 속성 / 이벤트

```
// 팩토리 — 채널별 연결
static Task<IksungReader> ConnectSerialAsync(string portName, int baudRate = 115200, CancellationToken ct = default)
static Task<IksungReader> ConnectSerialAsync(string portName, int baudRate, IksungReaderOptions options, CancellationToken ct = default)
static Task<IksungReader> ConnectPcscAsync (string readerName, IksungReaderOptions? options, CancellationToken ct = default)

// 속성
bool IsConnected { get; }
ChannelType ConnectedVia { get; }
IksungReaderOptions Options { get; } // ApplyOptions 로 변경 시 즉시 반영

// 이벤트
event EventHandler<TagDetectedEventArgs>? TagDetected // AutoRead 모드 카드 감지
event EventHandler<bool>? ConnectionChanged // true = 연결됨, false = 연결 해제
event EventHandler<RawPacketEventArgs>? RawPacketReceived // LogRawPackets=true 일

// 연결 생명주기 / 진단
void ApplyOptions (IksungReaderOptions options)
Task DisconnectAsync ()
Task<bool> PingAsync (int timeoutMs = -1, CancellationToken ct = default)
Task<ReaderInfo> GetReaderInfoAsync (int timeoutMs = -1, CancellationToken ct = default)

// Raw 명령 (전문가용)
Task<byte[]> SendRawCommandAsync (byte cmd1, byte cmd2, byte[]? data = null,
int timeoutMs = 1000, CancellationToken ct = default)

// 해제
ValueTask DisposeAsync()
```

IksungReaderOptions

```
public sealed class IksungReaderOptions
{
    bool AutoReconnect    { get; set; } // 기본 false - 비정상 끊김 시 자동 재연결
    int  ReconnectDelayMs { get; set; } // 기본 2000 - 재연결 시도 간격
    int  DefaultTimeoutMs { get; set; } // 기본 1000 - PingAsync 등 -1 일 때 사용
    bool LogRawPackets    { get; set; } // 기본 false - true 시 RawPacketReceived 이벤트
}
```

IksungReaderDiscovery (Serial 포트 탐색)

```
public static class IksungReaderDiscovery
{
    static Task<IReadOnlyList<string>> ScanIksungPortsAsync (
        int baudRate = 115200, int pingTimeoutMs = 300, CancellationToken ct = default)
    static Task<bool> PingPortAsync (
        string portName, int baudRate = 115200, int timeoutMs = 300, CancellationToken ct
    }
}
```

IksungPcscDiscovery (PC/SC 리더 탐색)

```
public static class IksungPcscDiscovery
{
    static IReadOnlyList<string> GetAvailableReaders()
    static void StartMonitor()
    static void StopMonitor()
    static event EventHandler<IReadOnlyList<string>>? ReaderListChanged
}
```

공통 명령 (MAJOR_COMMON = 0x00)


```
// 장치 정보
Task<string> ReadVersionAsync      (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadUniqueIdAsync    (int timeoutMs = 1000, CancellationToken ct = default)

// RF / 부저
Task          RfOnAsync           (int timeoutMs = 1000, CancellationToken ct = default)
Task          RfOffAsync          (int timeoutMs = 1000, CancellationToken ct = default)
Task          BuzzerAsync         (byte durationUnit = 2, int timeoutMs = 1000, CancellationToken ct = default)

// 카드 메타 (이미 활성화된 카드 대상)
Task<byte[]> ReadSakTypeAsync      (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadAtsAsync          (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadTagTypeAsync      (int timeoutMs = 1000, CancellationToken ct = default)

// 자동 카드 감지 / 종류 (카드 종류 무관하게 단일 호출)
Task<byte[]> ReadAllUidAsync       (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadAllCardTypeAsync (int timeoutMs = 1000, CancellationToken ct = default)

// 표준별 UID (Common 명령군)
Task<byte[]> ReadIso14443aUidAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadIso14443bUidAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadFelicaUidAsync    (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadIso15693UidAsync  (int timeoutMs = 1000, CancellationToken ct = default)

// 교통카드 일련번호
Task<byte[]> ReadTmoneySerialAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadCashbeeSerialAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadKcashSerialAsync  (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadRailPlusSerialAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadAllCashSerialAsync (int timeoutMs = 1000, CancellationToken ct = default)

// Watchdog Timer (WDT)
Task<byte[]> ReadWdtTimeoutAsync   (int timeoutMs = 1000, CancellationToken ct = default)
Task          WriteWdtTimeoutAsync (byte timeoutValue, int timeoutMs = 1000, CancellationToken ct = default)
```

ISO 14443 A/B (MAJOR_ISO14443AB = 0x01)

```
// 빠른 UID 읽기
Task<byte[]> ReadIso14443aUidAsync(int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ReadIso14443bUidAsync(int timeoutMs = 1000, CancellationToken ct = default)

// Layer-3 활성화
Task<byte[]> ActivateIso14443aAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ActivateIso14443bAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ActivateIso14443abAsync (int timeoutMs = 1000, CancellationToken ct = default)

// Layer-4 활성화 (ISO-DEP)
Task<byte[]> ActivateIso14443_4aAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> ActivateIso14443_3a4aAsync(int timeoutMs = 1000, CancellationToken ct = default)

// Halt
Task HaltIso14443aAsync(int timeoutMs = 1000, CancellationToken ct = default)
Task HaltIso14443bAsync(int timeoutMs = 1000, CancellationToken ct = default)

// APDU 교환
Task<byte[]> ExchangeApuAsync(byte[] apdu, int timeoutMs = 3000, CancellationToken ct = default)
```

Mifare Classic (MAJOR_MIFARE = 0x02)

```
Task<byte[]> ActivateMifareAsync(int timeoutMs = 1000, CancellationToken ct = default)

Task MifareAuthenticateAsync(
    byte blockNo, MifareKeyType keyType, byte[] key,
    int timeoutMs = 1000, CancellationToken ct = default)

Task<byte[]> MifareReadBlockAsync (byte blockNo, int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> MifareWriteBlockAsync(byte blockNo, byte[] data, int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> MifareReadSectorAsync(byte sectorNo, int timeoutMs = 1000, CancellationToken ct = default)
```

Mifare Ultralight / NTag (MAJOR_MIFARE_ULTRALIGHT = 0x05)

```

Task<byte[]> ActivateMifareUltralightAsync(int timeoutMs = 1000, CancellationToken ct = c
Task<byte[]> MifareUltralightReadPageAsync (byte page, int timeoutMs = 1000, Cancellatio
Task      MifareUltralightWritePageAsync(byte page, byte[] data, int timeoutMs = 1000,
Task      NTagPasswordAuthAsync          (byte[] password, int timeoutMs = 1000, Cancell
// NTag 전용
Task<byte[]> NTagGetVersionAsync          (int timeoutMs = 1000, CancellationToken ct = default
Task<byte[]> NTagFastReadAsync            (byte startPage, byte endPage, int timeoutMs = 2000,
Task<byte[]> NTagReadCounterAsync         (byte counterNo = 0, int timeoutMs = 1000, Cancellati
Task<byte[]> NTagReadSignatureAsync       (int timeoutMs = 2000, CancellationToken ct = default
Task      NTagWriteAuth0Async            (byte auth0, int timeoutMs = 1000, CancellationToken
Task      NTagWriteAccessAsync           (byte access, int timeoutMs = 1000, CancellationToken
Task      NTagChangePasswordAsync(byte[] newPassword, byte[] pack, int timeoutMs = 1000, Cancellatio

static string ParseNTagType(byte[] versionData)

```

ISO 15693 (MAJOR_ISO15693 = 0x03)

```

Task<byte[]> ReadIso15693UidAsync          (int timeoutMs = 1000, CancellationToken ct = default
Task<byte[]> ActivateIso15693Async         (int timeoutMs = 1000, CancellationToken ct = default
Task<byte[]> Iso15693ReadBlockAsync        (byte blockNo, int timeoutMs = 1000, CancellationToken
Task      Iso15693WriteBlockAsync         (byte blockNo, byte[] data, int timeoutMs = 1000, Cancellatio
Task<byte[]> Iso15693ReadMultipleBlocksAsync (byte firstBlock, byte blockCount, int timeoutMs = 1000, Cancellatio

```

DESFire (MAJOR_DESFIRE = 0x09)

```

// 활성화 / 정보
Task<byte[]> ActivateDesfireAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> DesfireGetFreeMemoryAsync(int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> DesfireGetUidAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireFormatAsync (int timeoutMs = 3000, CancellationToken ct = default)

// 키 관리
Task DesfireKeySaveAsync(byte keyIndex, byte keyVersion, byte cryptoType, byte[] key,
    int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireKeyInitAsync(int timeoutMs = 1000, CancellationToken ct = default)

// 인증
Task DesfireAuthenticateAesAsync (byte keyNo = 0, int timeoutMs = 2000, CancellationToken ct = default)
Task DesfireAuthenticate2K3DesAsync(byte keyNo = 0, int timeoutMs = 2000, CancellationToken ct = default)
Task DesfireAuthenticateIsoAsync (byte keyNo = 0, int timeoutMs = 2000, CancellationToken ct = default)
Task DesfireAuthResetAsync (int timeoutMs = 1000, CancellationToken ct = default)

// 애플리케이션
Task<byte[]> DesfireGetApplicationIdsAsync(int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireSelectApplicationAsync(byte[] appId, int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireSelectRootAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireDeleteApplicationAsync(byte[] appId, int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireCreateApplicationAsync(byte[] appId, byte keySettings, byte keyCount,
    int timeoutMs = 1000, CancellationToken ct = default)

// 파일
Task<byte[]> DesfireGetFileIdsAsync (int timeoutMs = 1000, CancellationToken ct = default)
Task<byte[]> DesfireGetFileSettingsAsync(byte fileNo, int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireCreateStdFileAsync (byte fileNo, byte commSettings, ushort accessRights,
    int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireDeleteFileAsync (byte fileNo, int timeoutMs = 1000, CancellationToken ct = default)

// 데이터 파일
Task<byte[]> DesfireReadDataFileAsync (byte fileNo, byte commSettings = 0, int offset = 0,
    int timeoutMs = 2000, CancellationToken ct = default)
Task DesfireWriteDataFileAsync(byte fileNo, byte[] payload, byte commSettings = 0,
    int timeoutMs = 2000, CancellationToken ct = default)

// 밸류 파일
Task<byte[]> DesfireReadValueFileAsync (byte fileNo, byte commSettings = 0,
    int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireCreditValueFileAsync(byte fileNo, int amount, byte commSettings = 0,
    int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireDebitValueFileAsync (byte fileNo, int amount, byte commSettings = 0,
    int timeoutMs = 1000, CancellationToken ct = default)

// 트랜잭션
Task DesfireCommitTransactionAsync(int timeoutMs = 1000, CancellationToken ct = default)
Task DesfireAbortTransactionAsync (int timeoutMs = 1000, CancellationToken ct = default)

```

LF 125 kHz (MAJOR_RF125KHZ = 0x0C)

```
Task<byte[]> ReadLf125KhzUidAsync      (int timeoutMs = 2000, CancellationToken ct = def
Task<byte[]> ReadLf125KhzRawUidAsync   (int timeoutMs = 2000, CancellationToken ct = def
Task<byte[]> ReadLfIso11784Async       (int timeoutMs = 2000, CancellationToken ct = def
Task<byte[]> ReadLfIso11784LowDataAsync (int timeoutMs = 2000, CancellationToken ct = def
Task<byte[]> ReadLfSecomBlockAsync     (int timeoutMs = 2000, CancellationToken ct = def
Task<byte[]> ReadLfRawBitsAsync        (byte bitLength = 64, int timeoutMs = 2000, Cance
Task<byte[]> ReadLfTemicBlockAsync     (byte readBits = 0x40, byte delayMs = 0x0A,
    int timeoutMs = 2000, CancellationToken ct = default)
Task<byte[]> ReadLfTemicLowDataAsync   (int timeoutMs = 2000, CancellationToken ct = def
Task<byte[]> ReadLfHtrcSamplingTimeAsync (int timeoutMs = 1000, CancellationToken ct = def
Task<byte[]> AutoTuneLfSamplingAsync   (int timeoutMs = 3000, CancellationToken ct = def
Task<byte[]> ReadLfTimingParamsAsync   (int timeoutMs = 1000, CancellationToken ct = def
```

AutoRead (MAJOR_AUTO = 0x20)

```
Task StartAutoReadAsync(int timeoutMs = 1000, CancellationToken ct = default)
Task StopAutoReadAsync (int timeoutMs = 1000, CancellationToken ct = default)
```

ISO 7816 / USIM (MAJOR_ISO7816 = 0x0A)

```
Task<byte[]> UsimActivateAsync (byte channel = 0, int timeoutMs = 2000, CancellationToke
Task          UsimDeactivateAsync(byte channel = 0, int timeoutMs = 1000, CancellationToke
Task<byte[]> UsimSendTpduAsync (byte[] tpdu, byte channel = 0, int timeoutMs = 3000, Can
Task<byte[]> UsimReadSerialAsync(int timeoutMs = 1000, CancellationToken ct = default)
```

Bluetooth / BLE (MAJOR_BLE_CONFIG = 0x12)

```

Task<string>   BleReadNameAsync           (int timeoutMs = 1000, CancellationTok
Task          BleWriteNameAsync          (string name, int timeoutMs = 1000, Cancell
Task<byte[]>   BleReadMacAddressAsync     (int timeoutMs = 1000, CancellationTok
Task<byte[]>   BleReadTxPowerAsync        (int timeoutMs = 1000, CancellationTok
Task          BleWriteTxPowerAsync        (byte powerIndex, int timeoutMs = 1000, Canc
Task<byte[]>   BleReadGapConnectParamsAsync (int timeoutMs = 1000, CancellationTok
Task          BleSystemResetAsync        (int timeoutMs = 1000, CancellationTok
Task<byte[]>   BleReadCentralEnableAsync  (int timeoutMs = 1000, CancellationTok
Task          BleCentralScanStartAsync    (int timeoutMs = 1000, CancellationTok
Task          BleCentralScanStopAsync     (int timeoutMs = 1000, CancellationTok
Task<byte[]>   BleCentralScanListAsync    (int timeoutMs = 2000, CancellationTok
Task          BlePeripheralAdvertisingStartAsync (int timeoutMs = 1000, CancellationToken
Task          BlePeripheralAdvertisingStopAsync (int timeoutMs = 1000, CancellationToken

static string TxPowerToString(byte powerIndex)

```

릴레이 보드 (MAJOR_RELAY = 0x22)

```

// 입력 읽기
Task<byte[]> RelayReadAllInputsAsync(int timeoutMs = 1000, CancellationToken ct = default
Task<byte[]> RelayReadInputAsync      (byte inputNo, int timeoutMs = 1000, CancellationTok

// 출력 읽기
Task<byte[]> RelayReadAllOutputsAsync(int timeoutMs = 1000, CancellationToken ct = default
Task<byte[]> RelayReadOutputAsync      (byte relayNo, int timeoutMs = 1000, CancellationTok

// 출력 제어
Task RelayWriteAllOutputsAsync(byte mask, int timeoutMs = 1000, CancellationToken ct = de
Task RelayWriteOutputAsync      (byte relayNo, bool on, int timeoutMs = 1000, CancellationT
Task RelayAllOffAsync           (int timeoutMs = 1000, CancellationToken ct = default)
Task RelayAllOnAsync            (int timeoutMs = 1000, CancellationToken ct = default)

// 설정
Task RelaySetAutoOffTimeAsync(byte relayNo, ushort timeMs, int timeoutMs = 1000, Cancell

// 유틸리티 (static)
static bool GetInputState(byte mask, byte inputNo)
static bool GetRelayState(byte mask, byte relayNo)

```

Raw 명령

```
Task<byte[]> SendRawCommandAsync(
    byte cmd1,
    byte cmd2,
    byte[]? data = null,
    int timeoutMs = 1000,
    CancellationToken ct = default)
```

모델 / 열거형

CardType

```
public enum CardType
{
    Unknown, Iso14443a, Iso14443b, Iso14443ab,
    Iso15693, MifareClassic, MifareUltralight,
    MifareDesfire, Felica, Lf125Khz
}
```

MifareKeyType

```
public enum MifareKeyType : byte { KeyA = 0x01, KeyB = 0x02 }
```

TagDetectedEventArgs

```
public sealed class TagDetectedEventArgs : EventArgs
{
    public CardType CardType { get; }
    public byte[] Uid { get; }
    public string UidHex { get; } // 하이픈 없는 대문자 16진수 (Uid 의 편의 변환)
    public byte Cmd1 { get; }
    public byte Cmd2 { get; }
    public byte[] RawData { get; }
}
```

RawPacketEventArgs

```
public sealed class RawPacketEventArgs : EventArgs
{
    public byte[]    Data        { get; }    // on-wire raw 바이트
    public bool      IsTransmit { get; }    // true=TX, false=RX
    public DateTime  Timestamp   { get; }
}
```

ReaderInfo

```
public readonly struct ReaderInfo
{
    public string      FirmwareVersion { get; }    // 예: "IS-3500K_V1.8"
    public ChannelType Channel         { get; }
    public bool        IsConnected    { get; }
    public string?     PortOrAddress  { get; }    // "COM3 @ 115200 bps" / "192.168.1.10"

    public override string ToString();
}
```

ChannelType

```
public enum ChannelType
{
    Serial,    // System.IO.Ports
    Ftdi,      // FTDI D2XX (예약 - 향후 채널)
    Pcsd,      // PC/SC (winscard / pcsc-lite)
    Socket,    // TCP/IP (내부 사용)
}
```

예외 (**Iksung.Reader.Exceptions** 네임스페이스)

```
IksungException           // 기반 (모든 SDK 예외의 부모)
ChannelDisconnectedException // 물리 채널 끊김 / 연결 실패
IksungTimeoutException    // 응답 타임아웃
IksungProtocolException    // 리더기가 STATE_FAIL 또는 깨진 응답 반환
```


.NET Framework 4.x 사용 가이드

Iksung.Reader SDK는 **.NET 8** 및 **.NET Framework 4.7.2** 를 모두 지원합니다.
이 문서는 .NET 4.x 환경에서 달라지는 사용 방법을 설명합니다.

지원 버전

플랫폼	최소 버전	용도
.NET 8	8.0	신규 개발 권장
.NET Framework	4.7.2	기존 WinForms/WPF 앱 통합

주요 문법 차이

1. 클래스·메서드 선언 필수 (Top-level statements 미지원)

.NET 8 예제:

```
// Program.cs - 클래스 없이 바로 코드 작성 가능
using Iksung.Reader;
var reader = await IksungReader.ConnectSerialAsync("COM3");
```

.NET 4.x 예제:

```
using System;
using System.Threading.Tasks;
using Iksung.Reader;

namespace MyApp
{
    internal class Program
    {
        static async Task Main(string[] args)
        {
            var reader = await IksungReader.ConnectSerialAsync("COM3");
        }
    }
}
```

2. `await using` → `try/finally + DisposeAsync()`

.NET 8:

```
await using var reader = await IksungReader.ConnectSerialAsync("COM3");
// 블록 종료 시 자동 해제
```

.NET 4.x:

```
var reader = await IksungReader.ConnectSerialAsync("COM3");
try
{
    // ... 작업 수행 ...
}
finally
{
    await reader.DisposeAsync(); // 명시적으로 해제
}
```

3. `using` 지시문 명시 필요

.NET 8 프로젝트는 `<ImplicitUsings>enable</ImplicitUsings>` 설정으로 `System`, `System.Threading.Tasks` 등이 자동으로 포함됩니다.

.NET 4.x 프로젝트는 필요한 `using` 을 모두 명시해야 합니다:

```
using System;
using System.Threading;
using System.Threading.Tasks;
using Iksung.Reader;
using Iksung.Reader.Exceptions;
```

4. 컬렉션 표현식 — 선택 사항

`LangVersion=latest` 를 설정하면 .NET 4.x에서도 C# 12 문법을 사용할 수 있습니다.

```
←!— .csproj →
<LangVersion>latest</LangVersion>
```

이 경우 아래 두 표현 모두 사용 가능합니다:

```
// C# 12 스타일 (LangVersion=latest 설정 시 net472에서도 가능)
byte[] key = [0xFF, 0xFF, 0xFF, 0xFF, 0xFF];

// 전통적인 스타일 (항상 가능)
byte[] key = new byte[] { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };
```

5. 이벤트 핸들러 등록 / 해제 명시

.NET 8 예제에서는 람다를 사용하는 경우가 많지만,
.NET 4.x에서는 해제를 위해 **이름 있는 메서드**를 권장합니다.

```
// ✅ 권장 — 해제 가능
EventHandler<TagDetectedEventArgs> handler = OnTagDetected;
reader.TagDetected += handler;
// ... 사용 후 ...
reader.TagDetected -= handler;

// ⚠️ 비권장 — 익명 람다는 해제 불가
reader.TagDetected += (s, e) => Console.WriteLine(e.UidHex);
```

6. WinForms — UI 스레드 마샬링 (필수)

`TagDetected` 이벤트는 **백그라운드 스레드**에서 발생합니다.
WinForms 컨트롤은 **UI 스레드에서만** 수정할 수 있습니다.

```
private void OnTagDetected(object? sender, TagDetectedEventArgs e)
{
    if (InvokeRequired)
    {
        // 백그라운드 스레드 → UI 스레드로 전환
        Invoke(new Action<TagDetectedEventArgs>(UpdateLabel), e);
        return;
    }
    UpdateLabel(e);
}

private void UpdateLabel(TagDetectedEventArgs e)
{
    lblUid.Text = "UID: " + e.UidHex;
}
```

7. WPF — Dispatcher 마샬링

```
private void OnTagDetected(object? sender, TagDetectedEventArgs e)
{
    Application.Current.Dispatcher.Invoke(() =>
    {
        lblUid.Content = "UID: " + e.UidHex;
    });
}
```

.NET 4.x 예제 목록

예제	경로	설명
HelloWorld	NET4x-Samples/HelloWorld.Console/	연결, 버전, 고유 ID
01-ReadAnyUid	NET4x-Samples/01-ReadAnyUid/	UID 폴링 (ISO14443A/B, ISO15693, LF)
02-Iso14443a	NET4x-Samples/02-Iso14443a/	ISO 14443-A Layer-3/4 + APDU
03-MifareClassic	NET4x-Samples/03-MifareClassic/	인증, 블록 읽기/쓰기
04-MifareUltralight	NET4x-Samples/04-MifareUltralight/	페이지 덤프/쓰기

예제	경로	설명
05-Iso15693	NET4x-Samples/05-Iso15693/	멀티블록 읽기
06-AutoRead	NET4x-Samples/06-AutoRead/	이벤트 기반 카드 감지
07-Desfire	NET4x-Samples/07-Desfire/	DESFire EV1/EV2 전체 워크플로
08-NTag213	NET4x-Samples/08-NTag213/	NTag213/215/216 버전/서명/카운터
09-Lf125KhzAdvanced	NET4x-Samples/09-Lf125KhzAdvanced/	EM410X, ISO11784, SECOM, Temic
10-Iso7816	NET4x-Samples/10-Iso7816/	USIM ATR + TPDU + ICCID 읽기
11-Bluetooth	NET4x-Samples/11-Bluetooth/	BLE 이름/MAC/TX파워/GAP 설정
12-Relay	NET4x-Samples/12-Relay/	릴레이 I/O 제어
13-CommandConsole	NET4x-Samples/13-CommandConsole/	대화형 RAW 명령 콘솔
WinFormsIntegration	NET4x-Samples/04-WinFormsIntegration/	WinForms Invoke 패턴
14-SystemFeatures	NET4x-Samples/14-SystemFeatures/	포트 탐색, 연결 옵션, ConnectionChanged, AutoReconnect, Raw 패킷

```
# 빌드
dotnet build NET4x-Samples/01-ReadAnyUid/ReadAnyUid.Net4x.csproj

# 실행
dotnet run --project NET4x-Samples/01-ReadAnyUid/ReadAnyUid.Net4x.csproj -- COM3
```

프로젝트 설정 참조 (.csproj)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net472</TargetFramework>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>disable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Iksung.Reader" Version="0.*" />
  </ItemGroup>
</Project>
```

[← 목차](#)
